UNIVERSITEIT VAN AMSTERDAM

Faculty of Economics and Business

In collaboration with Tencent Holdings Ltd.

LLM-Based Anomaly Detection in Live-Service Games

Evaluating Guided Analytics Agents on Game KPIs

MASTER'S THESIS
Data Science and Business Analytics

Daniel Osman

Student ID: 16062108

Supervised by,

Wenhao Chi

Tencent 腾讯

July 2026

Statement of Originality

This document is written by Daniel Osman, who declares that he takes full responsibility for its contents. I declare that the text and work presented in this document reflect my own efforts and conform to all applicable rules regarding academic integrity. All sources and tools used in its creation, including any permitted AI-based applications, have been appropriately acknowledged. UvA Economics and Business is responsible solely for supervising the completion and submission of this work, not for its contents.

Generative AI tools were used only for limited language-related support, including improving wording, grammar, structure, and readability, as well as assistance with LaTeX formatting. All research decisions, methodological choices, analysis, interpretation, and final content remain my own.

Abstract

Live-service game analytics requires the continuous monitoring of key performance indicators to identify unusual behavior, operational issues, and changes in player activity. This thesis investigates whether an LLM-based analytics agent can perform end-to-end anomaly detection on live-service game KPIs and whether structured methodological and game-specific guidance improves its performance. This addresses a gap in the literature, as anomaly detection research has focused far less on aggregate live-service game KPIs than on areas such as player-level behavior, cheating, or bot detection. To create a reliable basis for evaluation, the thesis first developed a verified statistical anomaly reference framework using approximately 180 days of Delta Force KPI data. The framework combined residual-based point detection, contextual anomaly detection, multivariate analysis, and changepoint and drift detection in order to identify the methods most suitable for this setting and produce an initial set of candidate anomalies. These candidates were then reviewed by Tencent analysts and domain experts to create the verified reference set used in the experiments. Three experiments were conducted: the first compared guided and unguided Claude Opus 4.8 runs, the second examined the stability of repeated guided Claude Opus 4.8 executions, and the third compared Claude Opus 4.8, DeepSeek-V4-Pro, and GLM-5.2 under the same guided setup. The results showed that the agent could carry out the anomaly-detection task end-to-end, but that additional context was important for achieving reliable performance. In Experiment 1, the guided condition increased micro F1 from 0.456 to 0.886, with improvements in both precision and recall, and the largest gains appeared during post-launch periods where contextual interpretation was especially important. Repeated Claude Opus 4.8 runs were generally stable, although some record-level variation remained during the more complex launch period. In the model comparison, Claude Opus 4.8 achieved the strongest overall performance across the selected LeoX workflow. Despite limitations related to the use of one game, a selected set of KPIs, a limited number of repeated runs, and incomplete transparency into the agents' analytical choices, the findings provide positive evidence that guided LLM agents can support live-game anomaly detection. The research was also used to develop a practical anomaly-detection layer within Tencent's internal environment. The workflow can be used across multiple supported LLMs and can analyze KPI data to generate structured reports for analyst review, helping to make the monitoring process more efficient and consistent. Overall, the thesis shows that LLM-based analytics agents can provide useful anomaly detection when supported by clear methodological guidance, an appropriate model, and continued human validation.

Contents

1 Introduction

Every video game generates a continuous stream of operational, behavioral, and transactional data on a second-by-second basis as players interact with a game and the system responds to those interactions. Almost every player action and system process inside the game can be recorded as an event, such as logging in, starting a session, joining matchmaking, completing a battle, making a purchase, or encountering a technical issue. The collected data is extremely useful for aggregating metrics such as daily active users, play times, revenue, paying users, registrations, matchmaking attempts, and matchmaking success rates. These metrics are usually monitored by analytics and operations teams because sudden changes, or anomalies, can indicate technical failures, monetization problems, player-experience issues, or unexpected behavioral shifts. At the same time, this information is also important for tracking key performance indicators and supporting business decisions. For example, changes in revenue, active users, or paying users can all be used in reports and discussions with executives and directors to understand whether a game is underperforming or overperforming. Therefore, anomalies are not only linked to problems, but can also represent positive changes that may require further analysis. Game updates, new seasons, patches, marketing campaigns, limited-time events, and weekly player cycles can all create large but expected changes in activity. As a result, anomaly detection in a live-service game analytics context is different from simply identifying values that are statistically far from the average. A value can only be interpreted as anomalous when it is compared against the correct expectation for that specific business and temporal context.

This thesis is conducted in collaboration with Tencent Holdings Limited and focuses on anomaly detection in production-level video game analytics. The research does not aim to build a general anomaly detection algorithm from scratch. Instead, it investigates how well an LLM-based analytics agent can perform anomaly detection when it is given relevant analytical context, including metric definitions, domain knowledge, methodological guidance, access to the required data, and the ability to run its own analytical code. This is relevant because generative AI systems are increasingly being used inside organizations to support business processes, improve productivity, and make access to information and data easier for employees [1]. For this reason, understanding how reliably these systems can carry out a specialized analytical task, rather than simply produce text as an output, is becoming increasingly more important.

At Tencent, an internal LLM-based analytics platform called LeoX has been developed to support employees in working with game and business data. This platform provides a user interface where employees can interact with different supported models, including GPT, Claude, Alibaba, and DeepSeek, depending on the task. The platform is mainly designed for analytics teams, business users, and Site Reliability Engineering (SRE) teams who need to work with game and operational data in a more accessible way. A central part of LeoX is that it is built directly for an internal company environment where data safety and controlled access are extremely important to maintain business confidentiality. This means that users can ask questions involving sensitive business data without directly exposing that data through public or uncontrolled tools. In practice, LeoX acts as a controlled layer between the user, the selected language model, and the internal analytics systems.

Normally, large language models do not have access to company databases, code environments, or internal software systems by themselves. They can generate text, code, or analytical reasoning, but they cannot interact with real data or execute tasks within real systems. LeoX is built to support this kind of interaction: it connects the selected model to a controlled set of internal analytics tools through tool connections such as MCPs, so the agent can request specific actions without being given direct or uncontrolled access to the underlying systems. In practice, a user may ask LeoX to retrieve a specific metric, inspect a table, generate a report, or run a query, and the connected tools execute the work through the backend systems while the model itself only creates the request. Depending on the game, dataset, or task, this may be handled by different internal data platforms, data warehouses, or query engines, such as Databricks, StarRocks, or Airflow. This research is not focused on explaining exactly how these systems work within the Tencent environment, since that information is mostly confidential. The relevant point is that LeoX already provides an analytics agent that can reason and run its own analytical code. Having access to a production-level agent at this scale, which is connected to real internal systems and live game data, is very rare outside of large companies. This makes LeoX a valuable setting for assessing what such an agent can actually do in practice. It also makes it possible to study whether the agent can carry out anomaly detection directly on real business and game data, rather than only in a controlled or synthetic environment.

Two gaps motivate this research. First, anomaly detection is well established in many domains, but its application to aggregate live game KPIs is less developed. Existing game-related anomaly detection research often focuses on player-level behavior, such as botting, cheating, or abnormal user behavior, rather than the operational and business-level metrics that analytics teams monitor every day. Second, although LLM agents can now retrieve data and execute code through internal tools, it is still not clear how reliably they can carry out a specialized analytical workflow such as anomaly detection. Having access to tools solves the access problem, but it does not automatically solve the methodological problem. The agent still needs a clear analytical procedure for deciding what data to use, which baseline is appropriate, which methods should be applied, and how the findings should be reported.

This thesis addresses these gaps by developing a structured approach for evaluating LLM-based anomaly detection in live-service game analytics. It first establishes a verified reference set of anomalies for the Delta Force KPIs, which provides a consistent basis for evaluating the later LLM outputs. It then tests how accurately and reliably an LLM-based analytics agent can reproduce these findings under different experimental conditions, including with and without additional methodological and game-specific context. The aim is therefore to assess not only whether the agent can identify plausible anomalies, but whether its outputs agree with a validated reference.

2 Related Work

2.1 Anomaly Detection in Time-Series Data

Anomaly detection is a mature field concerned with identifying observations or patterns that do not follow the expected behavior of a system. Chandola, Banerjee, and Kumar's paper [2] is one of the main foundations in this area as it defines different types of anomalies, including point anomalies, contextual anomalies, and collective anomalies. This distinction is important because an anomaly is not always just a single extreme value, especially in the context of live video-game KPIs. A point anomaly refers to one individual observation that is unusual compared to the rest of the data, such as one day where daily active users suddenly drops far below the expected level. A contextual anomaly is only unusual within a specific context. For example, a revenue spike during a major launch week may be normal, while the same spike during a calm mid-season week may require investigation. A collective anomaly occurs when a group of observations forms an unusual pattern, even if each individual value does not look extreme on its own. This can happen when a KPI gradually declines for several days or shifts into a new level after an update. This distinction is important for this thesis because the metrics monitored in a live game environment do not only contain isolated one-day spikes or drops. A sudden fall in daily active users may be treated as a point anomaly, but a persistent decline in revenue after a patch or update may be closer to a regime shift. Similarly, a launch-week increase in activity may only be abnormal if it is unusual compared to previous launch periods. These cases require different methods, baselines, and interpretation rules, and therefore cannot be handled reliably using only one generic outlier detector.

This distinction matters even more in time-series data, where observations are ordered and often depend on previous observations. Blázquez-García et al. [3] show that anomaly detection in time series is different from anomaly detection in static tabular data because the expected value of a point depends on its temporal context or previous behavior. In practice, this means that a value should not only be compared against the overall historical average, but against what would be expected at that point in time. This is directly relevant to video game KPIs because daily active users, revenue, paying users, registrations, and matchmaking rates can all contain weekly cycles, launch effects, trend changes, and event-related movements. A large increase on a weekend or during a new season may look unusual compared to the global average, but it may be normal for that context.

For this reason, preprocessing is an important part of anomaly detection in this thesis. STL decomposition, introduced by Cleveland et al. [4], separates a time series into trend, seasonal, and residual components. The residual component represents the part of the signal that remains after expected trend and seasonal structure have been removed. In the context of game analytics, this is useful because the goal is not to flag every large movement in a KPI, but to flag movements that are unexpected after accounting for normal weekly patterns and longer-term changes. Similar residual-based ideas have also been used in operational anomaly detection systems, where seasonal business or system metrics must be adjusted before abnormal behavior can be detected [5]. These methods

have also been used in cloud infrastructure and web-service monitoring, where application and system metrics contain regular seasonal patterns but still need to be monitored for abnormal behavior affecting performance, availability, capacity planning, and user behavior [5]. Hochenbaum et al. use seasonal decomposition to remove trend and seasonality before applying robust statistical detection, which supports the same general idea used in this thesis. This makes STL decomposition a suitable pre-processing step in the current context, especially when certain game KPIs show clear weekly seasonality.

Several detection methods are relevant after this preprocessing step. Z-score detection can be used as a simple statistical point-anomaly method by comparing target residuals against the residual distribution of a baseline period. Isolation Forest, introduced by Liu, Ting and Zhou [6], is an unsupervised anomaly-detection method that identifies observations that are easier to isolate from the rest of the data. In this thesis, both methods are applied to residuals obtained after STL decomposition rather than directly to raw KPI values. This helps prevent expected trend and weekly seasonal movement from being incorrectly treated as anomalous. These methods are especially useful for detecting sudden one-day spikes or drops, such as a sharp fall in paying users or an unexplained increase in matchmaking failure rate.

However, point-anomaly methods are not sufficient for all relevant cases. Some important KPI changes are not visible as one extreme value, but instead appear as a shift in the behavior of the series. When a metric moves into a new level, changepoint detection becomes more appropriate than simple point detection. PELT, introduced by Killick, Fearnhead and Eckley [7], is designed to identify changepoints efficiently over a full time series. This is also relevant as game KPIs can shift after updates, launches, or technical changes, and the beginning of that shift may be more important than any single value inside the shifted period.

Drift detection is also relevant because some changes happen gradually rather than suddenly. A slow decline in registrations, paying users, or matchmaking quality may not trigger a point-anomaly detector on any individual day, but the overall movement may still be meaningful from a business or operational perspective. CUSUM is one classical approach for this type of problem because it accumulates deviations from an expected level over time and can therefore detect persistent shifts that are not obvious from isolated observations [8].

A further issue in live-game data is that some anomalies are contextual rather than purely statistical. This is where season and event comparison become important. If a game update, new season, or promotional event is expected to increase activity, then a high value during that period should not automatically be treated as abnormal. Instead, it should be compared against a more appropriate reference period, such as a similar launch window or equivalent point in a previous season. This follows the logic of contextual anomaly detection described by Chandola et al. [2], where the same value may be normal in one context but anomalous in another. It is also consistent with the broader idea of seasonal adjustment in time-series analysis, where recurring patterns are removed or accounted for before analyzing the remaining movement. In the game setting, this is especially important because in-game events and promotions are deliberately used to affect player activity and retention [9]. Therefore, comparing a launch week only against calm historical weeks would risk treating expected launch behavior as anomalous.

This is the main reason why the current thesis does not treat live-game anomaly detection as a single-method problem. The relevant KPI movements can appear as point spikes, contextual launch-period deviations, gradual drift, changepoints, or cross-metric patterns. Existing anomaly detection research provides methods for each of these cases, but there is limited research that combines them specifically for aggregate live-game KPI monitoring. The framework in this thesis therefore uses the related work as a basis for combining residual-based point detection, launch-aware season comparison, multivariate checks, and changepoint detection into one evaluation framework for live game analytics.

2.2 Anomaly Detection in Games

Anomaly detection has also been applied in game-related research, but much of this work focuses on player-level behavior rather than operational KPI monitoring. For example, Kang et al. [10] study game bot detection using user behavioral characteristics in an MMORPG setting. This type of work is important because it shows that game telemetry can be used to detect abnormal behavior, especially when users behave in repetitive or suspicious ways. Other game-related studies have also focused on cheating, botting, abnormal player behavior, or player profiling. These studies are useful for understanding how anomaly detection can be applied to games, but they

do not directly solve the problem studied in this thesis.

The setting of this thesis is different because it focuses on aggregate live-service KPIs rather than individual player behavior. The goal is not to decide whether a single player is cheating or behaving abnormally, but whether the overall game is experiencing unusual movement in metrics such as daily active users, revenue, paying users, registrations, and matchmaking quality. These metrics are used by analytics and operations teams to understand whether the game is healthy. They are also affected by game-specific business context, such as new seasons, patches, updates, marketing activity, holidays, and recurring weekly player cycles. This makes the problem different from many existing game anomaly detection studies because the expected behavior of the metric depends heavily on the operational context of the game.

This point is especially important in live-service games because game companies regularly use updates, events, and promotions to affect player activity and retention. Kim and Kim [9] note that online game companies carry out events to attract new users, maximize returning users, and reduce churn, and they study how users respond differently to in-game promotion events. This supports the idea that event periods can naturally change player behavior and therefore change the KPI baseline. In other words, not every large movement after an update is an anomaly. A new season may increase DAU, revenue, and registrations for expected reasons, while a payment issue or matchmaking failure during the same period may still be anomalous. The statistical problem is therefore not only to detect movement, but to decide whether the movement is unexpected for the current game context.

This creates a specific research gap. Game anomaly detection has been studied, but the live KPI monitoring problem remains less explored. A game KPI can move sharply for completely normal reasons, especially during a launch week or after a major update. At the same time, smaller movements may be important if they occur in the wrong metric or direction, such as a drop in matchmaking success rate or a fall in paying users after an update. Therefore, anomaly detection in this setting requires both statistical treatment of the time series and domain-aware interpretation of the game context. This is one of the reasons why the framework in this thesis combines residual-based detection, season comparison, multivariate checks, and changepoint detection rather than relying on a single detector.

2.3 Large Language Models for Time-Series Anomaly Detection

Recent research has started to examine whether large language models can be used for time-series anomaly detection. Alnegheimish et al. [11] investigate whether LLMs can act as anomaly detectors for time series by converting time-series data into text and prompting models to identify anomalous values. Their results show that LLMs are capable of detecting some anomalies, but also that they are not automatically superior to specialized anomaly detection models. This is directly relevant to the present thesis because it supports the idea that LLMs may be useful in anomaly detection, but should not be treated as reliable detectors without careful structure, representation, and evaluation.

Other work reaches a similar conclusion. Dong, Huang and Cao [12] argue that LLMs cannot simply be used directly as time-series anomaly detectors without appropriate prompting and task design. Zhou and Yu [13] also study whether LLMs can understand time-series anomalies and show that model behavior varies across architectures and that the representation of the time series has a major impact on performance. These findings are important because they suggest that the problem is not only whether an LLM is powerful enough, but whether the task is presented in a form that the model can handle reliably. In this thesis, this is addressed through structured context guidance. The agent is provided with theoretical and methodological information about anomaly types, temporal baselines, seasonality, game updates, and potentially useful detection methods.

Existing work on LLMs for time-series anomaly detection usually evaluates the model on benchmark datasets or converted time-series inputs. This thesis instead evaluates an LLM-based analytics agent in a production-level environment, where the agent can access data, read Skills, execute Python code, and generate analytical outputs. The task is therefore not only to identify anomalous values from a sequence, but to carry out a full analytics workflow by running code and examining real data. A model may be able to reason about anomalies in a prompt, but that does not mean it will consistently execute a correct anomaly detection workflow when operating as an analytics agent.

2.4 LLM Agents and Structured Context Guidance

The ReAct framework introduced by Yao et al. [14] is also relevant because it describes how language models can combine reasoning with actions by interacting with external tools or environments. This matters for the current thesis because LeoX is not only used as a text-generation system. It can read internal Skills, generate SQL, request query execution through controlled tools, run Python code, and return structured analysis. In other words, LeoX can perform parts of an analytics workflow rather than only explain what an analyst should do. This makes it possible to study anomaly detection as an agentic analytics task instead of as a simple text-generation task.

However, giving an LLM access to tools does not automatically define how it should carry out a specialized analytical task. The agent may be able to retrieve data, write code, and produce a report, but it still needs to make methodological decisions about the analysis. For anomaly detection, this includes deciding how much historical data is needed, what baseline should be used, whether seasonality should be accounted for, which detection methods are appropriate, and how the findings should be reported. This makes anomaly detection a useful test case for studying whether an LLM agent can carry out a specialized analytics workflow when it is given more task-specific methodological context.

In this thesis, that methodological context is provided through LeoX Skills. Skills are reusable instruction files that define task-specific knowledge and workflows for the agent. For anomaly detection, the Skill is designed using two sources of information: the general methods and principles identified in the related work, and the game-specific context found to be relevant during the development of the local framework. The Skill explains the anomaly detection problem, the behavior of live-game KPIs, and the types of methods that may be useful, but it does not define a fixed implementation or provide the verified anomaly labels. It therefore supports the agent's reasoning without determining the final analytical procedure or output.

2.5 Research Gap

The literature therefore leaves three gaps that this thesis addresses. First, anomaly detection methods are well established, but live video game KPIs create a difficult setting because the data is seasonal, non-stationary, and heavily affected by game events, launch periods, updates, and player cycles. Second, game anomaly detection research often focuses on player-level behavior, such as cheating or botting, rather than aggregate operational and business KPIs used by analytics teams. Third, recent work shows that LLMs can sometimes detect time-series anomalies, but their reliability as tool-using analytics agents in a real company environment remains largely untested.

This thesis sits at the intersection of these gaps. It does not propose a new anomaly detection algorithm. Instead, it builds a verified statistical reference framework for live-game KPIs and uses this reference to assess whether an LLM-based analytics agent can carry out anomaly detection end to end. It then examines whether providing structured methodological knowledge and game-specific context improves the agent's outputs enough to make the workflow more accurate, consistent, reliable, and practically usable. The reference framework is essential because the research question cannot be answered by assessing whether LeoX produces a convincing anomaly report. Its outputs must be compared against a controlled reference that defines what should have been detected. The contribution is therefore both methodological and practical, as it evaluates the feasibility and dependability of guided LLM-based anomaly detection in a real company environment.

3 Research Question Primary Focus

Building on the gaps identified in the previous sections, this thesis focuses on whether an LLM-based analytics agent can detect anomalies within video game KPIs and whether additional methodological context improves its performance. The central research question and its subquestions are stated below.

3.1 Research Question

"How well can an LLM-based analytics agent detect anomalies in live-service game KPIs, and to what extent does methodological context improve its accuracy, consistency, and reliability relative to a verified statistical

reference?"

3.2 Subquestions

1. Which anomaly-detection methods and methodological choices are suitable for constructing a verified statistical reference for live, non-stationary, and seasonal game KPI data?
2. How accurately can an LLM-based analytics agent detect anomalies compared to the verified reference standard, in terms of precision, recall, F1, false positives, and false negatives?
3. To what extent does structured methodological guidance improve the agent's anomaly detection performance, consistency, and reliability, including method selection, baseline choice, output structure, and repeated-run stability?
4. How does anomaly detection performance vary across the different LLM models available in LeoX when given the same data, task, and structured guidance?

Together, these subquestions build up to an answer for the main research question. The first establishes a suitable combination of methods and methodological choices for building the local reference framework. The second evaluates whether LeoX can correctly identify anomalies relative to that verified reference. The third tests whether the agent's behavior is reliable across repeated runs and output requirements. The fourth benchmarks the different models available in LeoX to see how much performance depends on the underlying model. From an academic perspective, answering these questions extends anomaly detection research into the understudied setting of aggregate live-game KPI monitoring and contributes to understanding how well LLM agents can carry out practical analytical workflows. From a company perspective, it provides Tencent with evidence on whether an anomaly detection workflow can be embedded into LeoX through a Skill and which model and guidance setup produces the most dependable results.

4 Methodology

4.1 Data Setting and KPI Construction

The data used for this thesis comes from a game called Delta Force, which is a live-service tactical first-person shooter operated within Tencent's game environment. Delta Force is a multi-platform game available across PC and mobile platforms, with console support also introduced as part of the wider product rollout. The game is developed by Team Jade and published under Tencent's TiMi Studio Group, and it operates across international regions rather than being a small single-market title. This makes it a suitable setting for this thesis because a game of this scale produces continuous behavioral, transactional, and operational data across different platforms, regions, player segments, and game systems.

Delta Force averages approximately 120,000 daily active users on Steam alone [15]. In a live-service game, almost every meaningful player action and system interaction is recorded as an event. This includes actions like logging in, registering, entering matchmaking, completing sessions, making purchases, receiving rewards, changing loadouts, and interacting with different game systems. At the scale of Delta Force, these event logs can amount to billions of rows per day. However, working directly with that level of raw event data is difficult and too detailed for weekly anomaly detection. For this reason, these events are aggregated into daily KPIs and are used by analytics, operations, and site reliability engineering teams to monitor and understand game performance, player behavior, monetization, and operational health. The dataset used in this thesis was constructed by aggregating these events into daily KPI values for Delta Force. The final dataset covers approximately 180 days, from November 2025 to May 2026, and is used as the main input for the local anomaly detection framework. This level of aggregation is suitable for the thesis because the goal is not to detect individual abnormal player actions but to detect unusual movements in the operational and business metrics that teams use to monitor the health of the game.

The main metrics collected within this dataset are daily active users, daily revenue, paying users, new registrations, matchmaking success rate, and matchmaking failure rate. These metrics were selected because they cover the main operational and business dimensions of almost all live service games. Daily active users and new registrations capture player activity and acquisition. Daily revenue and paying users capture monetization.

Matchmaking success and failure rates capture an important operational quality dimension, because a matchmaking issue can directly affect player experience even if business metrics do not immediately move.

The aggregation from raw event data to daily KPIs is also very important in the context of this research. This choice makes the problem closer to how game analytics teams actually monitor game health. Teams usually need to know whether a KPI was unusual on a given day or during a given week, rather than inspect every individual event. Daily aggregation also makes the analysis more computationally practical, because many anomaly detection methods require historical windows and repeated comparisons over time. Running these methods directly on unaggregated event-level data would be extremely expensive, slower, and less aligned with the business reporting layer used by analysts.

4.2 Ground Truth Problem and Reference Construction

A key challenge in this thesis that must first be addressed before attempting to answer all of the research questions is that the Delta Force KPI dataset does not come with pre-labeled anomalies. The data shows how each metric moved over time, but it does not state which dates and metrics should be treated as true anomalies. This creates a problem for evaluation because if an LLM produces anomalies for a specific period, its outputs cannot be verified or evaluated unless there is a reference set that defines which anomalies should have been detected. It needs to be compared against a reference that defines what should have been detected. Without such a reference, it would not be possible to calculate precision, recall, false positives, or false negatives, and the evaluation would become mainly subjective. This issue is especially important because, as discussed in the related work, anomaly detection in live-service game KPI data is not a solved or standardized problem. Existing anomaly detection research provides useful methods for point anomalies, contextual anomalies, changepoints, drift, and residual-based detection, but it does not directly define which combination of methods should be used for aggregate live game KPIs.

This connects directly to the first subquestion of the thesis: which anomaly detection methods are most suitable for live, non-stationary, and seasonal game KPI data, and what preprocessing, baseline selection, thresholds, and parameters are required to create a verified statistical reference? Before an LLM can be evaluated, the thesis first has to answer this methodological subquestion. The ground truth is therefore not assumed to exist. It is constructed through a Delta Force anomaly detection framework that is based on the related work and the research conducted during the project, adapted to the behavior of the Delta Force KPI data, and then checked using expert knowledge from Tencent. This step is necessary because the thesis needs a defensible reference for what should count as an anomaly before any LLM output can be scored. The detailed choices behind the framework, including regime classification, expected-value construction, residual detection, threshold calibration, and changepoint detection, are explained in the following methodology sections.

Because there was no single standardized anomaly-detection workflow for this specific live-game KPI setting, the existing monitoring process depended heavily on manual review of dashboards, presentations, and individual KPI movements. Different analysts could use different comparison periods, analytical methods, thresholds, and contextual information depending on the metric, game period, and purpose of the analysis. Although this flexibility allowed analysts to apply their domain knowledge, it also created consistency problems because the same KPI movement could be flagged or interpreted differently by different people. This issue was further complicated by the limited research on anomaly detection for aggregate live-game KPIs, which meant that there was no single established method that could be adopted directly as a consistent reference. For this reason, the thesis first builds a Delta Force anomaly-detection framework before running the main evaluation experiments. The purpose of the framework is not only to automate part of this previously manual analytical process, but also to create a more structured and reproducible reference against which the LLM outputs can be evaluated. The framework combines methodological principles from the related work with practical knowledge gained from analyzing this type of data. The related work informs the statistical part of the framework, including residual-based point detection, contextual launch comparison, multivariate anomaly detection, changepoint detection, and drift detection.

The framework then produces a structured set of candidate anomaly records containing the date, metric, anomaly type, direction, observed value, expected value, and supporting detection methods. These candidate records are not automatically treated as the final ground truth. The framework code, parameter choices, methods, and detector logic were first reviewed by a Tencent analyst with experience working with the relevant game data.

The candidate anomalies were then separately reviewed by multiple Tencent analysts and domain experts using internal presentations, dashboards, and operational information that were not available within the research dataset. After this review, the accepted records become the verified ground-truth set used in the experiments. An LLM anomaly is counted as a true positive when it matches a verified reference anomaly, as a false positive when it reports an anomaly that is not in the reference, and as a false negative when it misses a verified anomaly. This process makes it possible to evaluate the accuracy, consistency, and reliability of the LLM outputs against a reference that is both statistically constructed and checked against the wider game context. The detailed validation and finalized procedure is explained later in the methodology section.

4.3 Research Design

Based on the ground-truth problem described above, the methodology is organized into two connected stages. The first stage builds the verified statistical reference used for evaluation. This stage starts with the daily Delta Force KPI dataset and the game-specific context needed to interpret it, such as metric definitions and the season calendar that includes the dates of each update that may impact these KPIs. The local reference framework then classifies each target period, constructs expected values, applies the anomaly detection methods, and consolidates the detected candidates into structured anomaly records. These records are then reviewed before being treated as the verified ground truth for the thesis.

The second stage evaluates LLM-based anomaly detection against this verified reference. After the reference anomaly set has been created, the same anomaly detection tasks are repeated under different experimental conditions. In the unguided condition, the LLM is asked to identify anomalies without access to the anomaly detection Skill. In the guided condition, the LLM is given the same type of task, but with methodological context provided through the Skill. This allows the evaluation to compare outputs produced with and without structured methodological guidance. The detailed experimental setup, including the selected evaluation periods, prompts, models, and scoring procedure, is explained in the later evaluation sections. The outputs from these runs are then compared against the verified reference using precision, recall, F1, false positives, and false negatives. This makes it possible to evaluate whether the LLMs actually detected the correct anomalies, whether they are consistent, and which LLMs perform best at this task.

Figure 1 summarizes the full research design. The pipeline starts with the Delta Force KPI dataset and the game-specific context needed to interpret it, such as the metric definitions and event dates that impact the KPIs. These inputs are used by the Delta Force anomaly detection framework to classify the analysis period, construct expected values, run the detection layers, and produce candidate anomaly records. The candidate records are then manually reviewed with Tencent experts. After this validation step, the confirmed records become the verified anomaly reference set. The second part of the pipeline uses this verified reference set to evaluate LLM-based anomaly detection. The same anomaly detection tasks are run under different conditions, including an unguided condition and a guided condition where methods are provided as context to the LLM as a Skill. The LLM outputs are then matched against the verified reference set and scored using true positives, false positives, false negatives, precision, recall, and F1. As explained in the introduction, LeoX provides the platform through which the LLMs can access Skills, data tools, query execution, and Python analysis. The following subsections explain the main components of this pipeline in more detail.

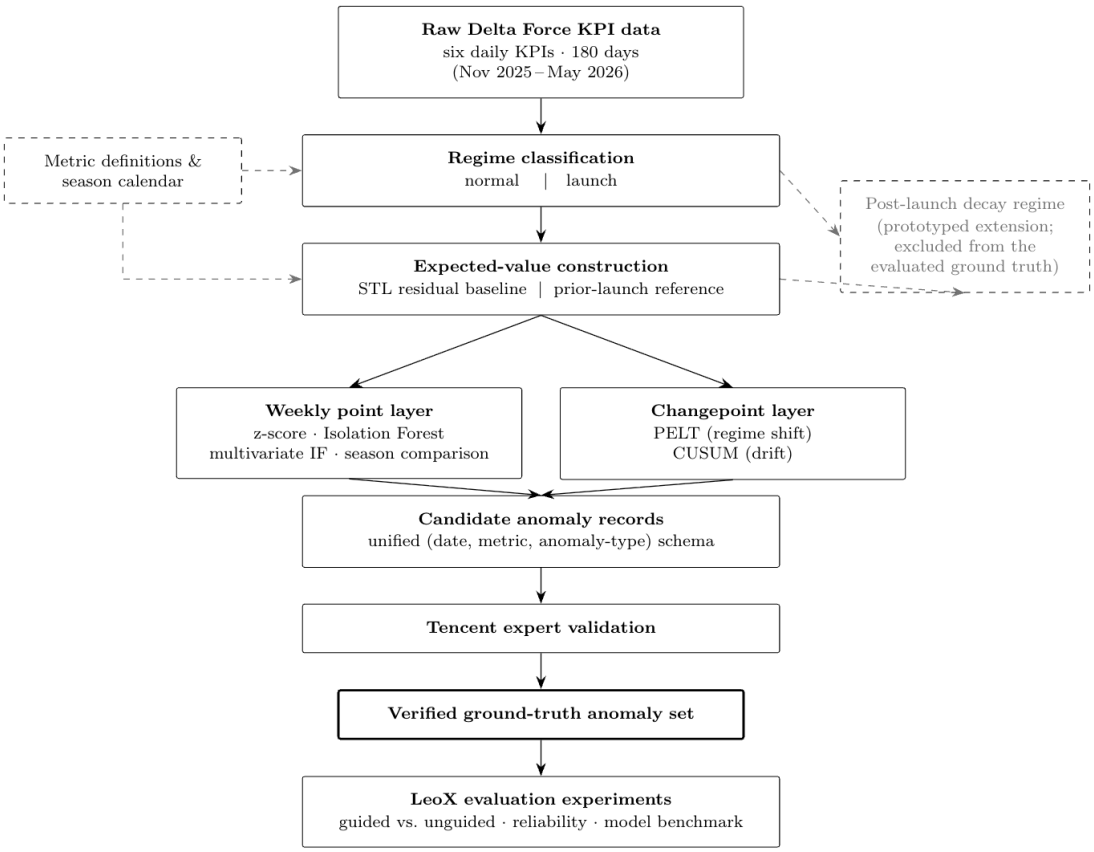

Figure 1: Overview of the methodology pipeline for constructing the verified anomaly reference set and evaluating LLM outputs against it.

The next subsections explain the first stage of this pipeline in more detail. They explain how the Delta Force anomaly detection framework classifies periods, constructs expected values, detects anomalies, and turns those detections into a verified reference set.

4.4 Framework Overview: Regimes and Detection Layers

The Delta Force anomaly detection framework is organized around two design choices. First, each analysis period is interpreted in relation to the game calendar before anomaly detectors are applied. Second, anomaly detection is split into two detection layers: a point-anomaly layer and a longer horizon changepoint layer. The point-anomaly layer includes both univariate detectors, which evaluate individual KPIs, and a multivariate detector, which checks whether the combination of KPI movements on a given day is unusual. The changepoint layer then captures longer-term shifts and sustained drift. This structure follows from the related work discussed earlier, where anomalies may appear as individual point deviations, contextual anomalies, collective patterns, or structural changes over time [2]. For this reason, the framework needs to consider not only the size of a KPI movement but also when it occurs and what game context surrounds it. It also needs to detect both isolated daily deviations, unusual cross-metric patterns, and longer changes in KPI behavior.

The first step of the framework is regime classification. In live-service game data, the same KPI movement can have a different interpretation depending on whether it occurs during ordinary operation or around a known game update. This follows the idea of contextual anomalies, where a value is only unusual relative to the context in which it occurs [2]. For this reason, the framework does not apply identical comparison logic to every period. In the evaluated framework, the two core regimes are normal and launch. A launch regime refers to an evaluation period associated with a major Delta Force season update, when player activity, acquisition, and monetization may be expected to move more strongly than during ordinary operation. These periods are identified using the

Delta Force season-update calendar rather than inferred from the KPI values themselves.

Periods not classified as launch periods are treated as normal evaluation periods. This does not mean that they are always fully stable, because some may still contain post-launch effects or other changes in player behavior. Instead, it means that they are evaluated against recent historical behavior and the weekly pattern of the metric rather than through the launch-specific comparison. Post-launch periods remain particularly important because KPI values may still be affected by an earlier release while beginning to move toward a more stable level. These periods are therefore interpreted using the recent trend, available post-launch history, weekly behavior, and wider contextual information rather than being treated as part of a fixed launch window.

After the regime is determined, the framework applies two detection layers. The point-anomaly layer evaluates daily KPI movements against expected values and identifies short-term deviations in individual metrics. This layer is designed for sudden spikes or drops. The changepoint layer analyzes a longer time series to detect structural shifts and sustained drift. This is necessary because some KPI changes do not appear as one extreme day but instead as a new level or gradual movement over time. Changepoint and drift methods are therefore included because point-anomaly methods alone are less suitable for changes in the underlying behavior of the series [7, 8].

4.5 Expected-Value Construction and STL-Based Residual Detection

Before applying any anomaly detectors, it is important to define what the detectors should be applied to. In time-series anomaly detection, the raw observed value is not always the right object to evaluate because it may contain expected temporal structure. As discussed in the related work, time-series observations depend on their temporal context. This means that the same value can be normal or abnormal depending on when it occurs, and should therefore be judged relative to what was expected at that point in time [3]. This is very important for live-game KPIs, because daily values can reflect trends, weekly player behavior, and unexplained movement at the same time. For example, DAU, registrations, or revenue may naturally increase during weekends because more players are active. If detectors are applied directly to raw KPI values, normal weekly cycles or gradual changes in the player base may be incorrectly treated as anomalies.

Figure 2 shows this weekly expected movement for daily active users. Each week is normalized to its own weekly mean so that the figure highlights the shape of the weekday and weekend pattern. The recurring weekday and weekend pattern provides a clear example of why expected weekly behavior should be accounted for before anomaly detection.

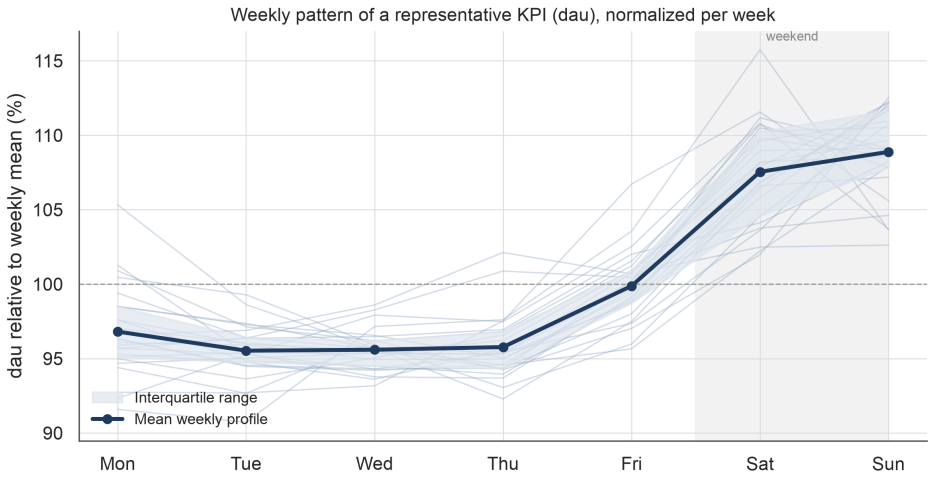

Figure 2: Weekly pattern of DAU, normalized to each week's mean. The figure shows a recurring weekday–weekend structure.

For this reason, the framework uses STL decomposition as the main preprocessing method for metrics with clear

weekly patterns. STL separates an observed time series into trend, seasonal, and residual components [4]. In this thesis, the seasonal period is set to 7 because the data is daily and player behavior often follows weekly rhythms. The point-anomaly detectors are then applied to the residual component rather than the raw KPI value. This means that the framework is not looking for the largest raw movement, but for the movement that remains unexpected after accounting for trend and weekly seasonality. This can be seen in:

$$y_{t,m} = T_{t,m} + S_{t,m} + R_{t,m} \tag{1}$$

where $y_{t,m}$ is the observed value of metric m on day t, $T_{t,m}$ is the trend component, $S_{t,m}$ is the weekly seasonal component, and $R_{t,m}$ is the residual component.

A seasonal period of seven does not mean STL is fitted to a single week. The decomposition is fitted over the full baseline history preceding the target period, and the value seven only specifies the length of the repeating cycle that is removed. For each target day, the expected value is the sum of the projected trend and the projected weekly seasonal component, and the residual is the difference between the observed and expected value. This can be seen in:

$$\hat{y}_{t,m} = \hat{T}_{t,m} + \hat{S}_{t,m} \tag{2}$$

$$r_{t,m} = y_{t,m} - \hat{y}_{t,m} \tag{3}$$

where $\hat{y}_{t,m}$ is the expected value and $r_{t,m}$ is the residual used by the residual-based detectors. The trend part of the equation is based on a linear slope estimated from the last fourteen fitted trend values. Using a linear model helps simplify the process and avoids introducing an additional forecasting model whose assumptions would be harder to evaluate and reproduce. The goal is also to just create a stable expected-value baseline for anomaly detection, which is why it is not necessary to have the most optimal forecast model. The trend projection can be seen as:

$$\hat{T}_{t+h,m} = T_{t,m} + h \cdot \hat{\beta}_m \tag{4}$$

where h is the number of days ahead in the target period and $\hat{\beta}_m$ is the linear trend slope estimated from the last fourteen fitted trend values for metric m. The period being checked for anomalies is kept separate from the historical baseline used to build expectations. This means that the framework only uses data before the start of the target period to estimate the trend, seasonal pattern, residual distribution, and detector baselines. The target period itself is then scored against this pre-built baseline. This prevents information from the period being evaluated from leaking into the expectation used to judge it. Figure 3 gives a visual example from the Delta Force registrations metric, showing how the observed value is compared against the expected value and how the residual is used to flag anomalies.

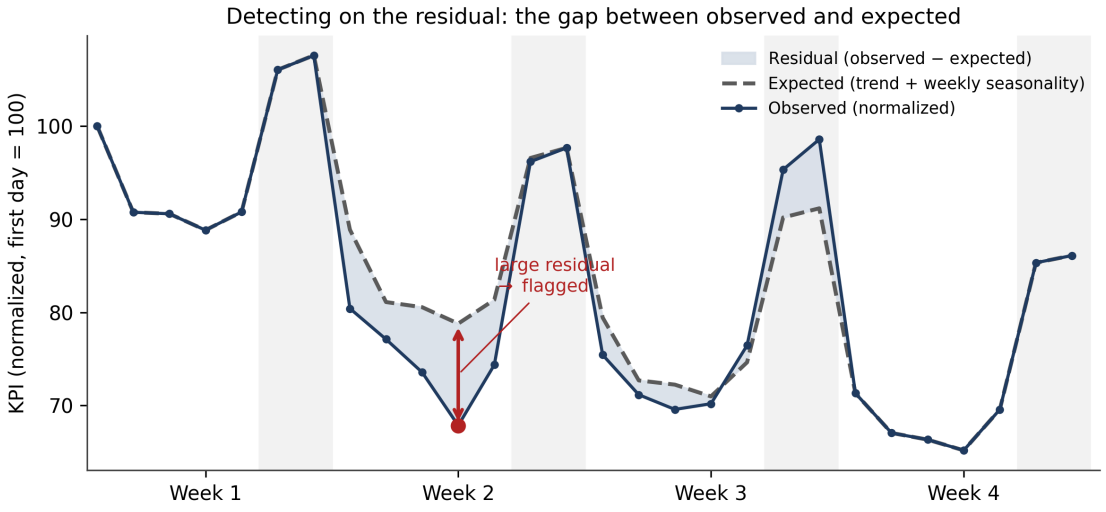

Figure 3: STL example using the registrations metric

It is also important to note that STL is not applied unconditionally. Because STL-based residual detection is only useful when a metric has a meaningful weekly pattern, the framework first checks whether the metric has enough baseline history and whether its lag-7 autocorrelation is above a practical threshold of 0.30. The lag-7 autocorrelation measures whether values tend to resemble the same weekday one week earlier. When this condition is not met, the framework falls back to a seven-day rolling-median baseline instead of forcing STL decomposition. The rolling median is used because it provides a simple local expected value without assuming a stable weekly seasonal pattern. It is also more robust to isolated spikes than a rolling mean, which is useful in anomaly detection because one unusual day should not strongly distort the entire baseline used to judge nearby days. Figure 4 shows the rolling lag-7 autocorrelation distribution for each monitored KPI and illustrates why the STL decision is made per metric rather than assumed for all metrics.

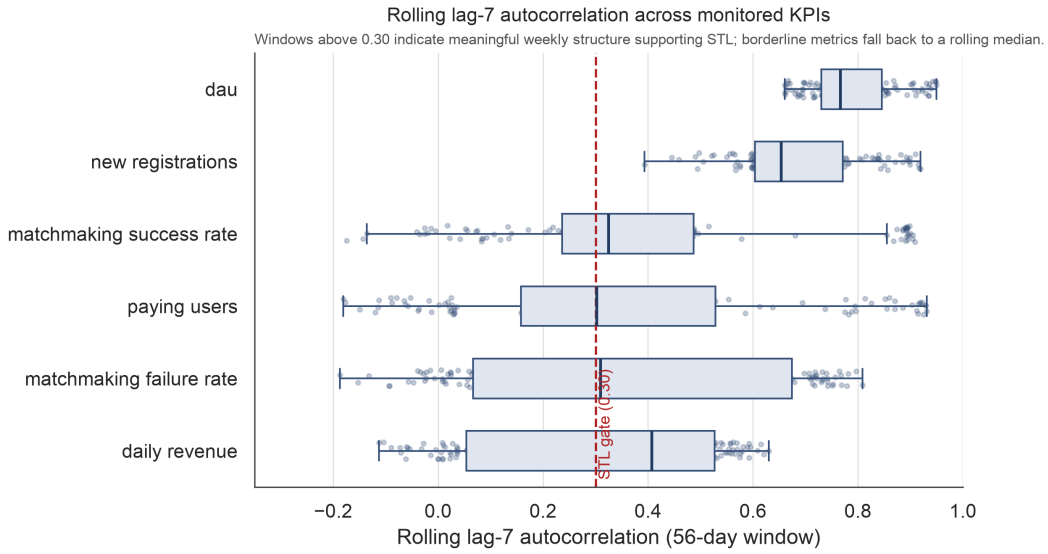

Figure 4: Rolling lag-7 autocorrelation across the monitored KPIs. The dashed line marks the STL threshold at 0.30.

4.6 Point-Anomaly Detectors

The point-anomaly section of the framework uses four detectors: Z-score detection, univariate Isolation Forest, season comparison, and multivariate Isolation Forest. Z-score and univariate Isolation Forest are applied to the residuals produced by STL when the metric has sufficient weekly structure. When STL is not appropriate, the same detectors are applied to deviations from the rolling-median expected-value baseline. These detectors are used together because game KPI anomalies can appear in different forms. Some anomalies are single-day deviations in one metric. Some are unusual only when compared with an equivalent launch-relative day from a previous season. Others are not extreme in any one metric but are unusual as a combination across several KPIs.

Z-score detection is the simplest residual-based point detector in the framework. It measures how far a target residual is from the baseline residual distribution in standard deviations. Because the method is applied to residuals rather than raw values, it does not treat expected weekly or trend-related movement as anomalous. Residuals with an absolute Z-score of at least 2.5 were treated as candidate detection signals, while values of at least 3.0 represented stronger statistical evidence.

$$z_{t,m} = \frac{|r_{t,m} - \mu_{r,m}|}{\sigma_{r,m}} \tag{5}$$

where $\mu_{r,m}$ and $\sigma_{r,m}$ are the mean and standard deviation of the baseline residuals for metric m.

Univariate Isolation Forest is also applied to residuals for each metric separately. The model is fitted on the baseline residuals and then used to score the target residuals. Isolation Forest is useful because it can identify

observations that are easier to isolate from the baseline distribution, even when the residual distribution is not perfectly normal. The contamination parameter is set to 0.02, meaning the method is configured to expect a small proportion of anomalous points. A fixed random seed is used so that repeated runs on the same data are reproducible.

The framework does not save a candidate anomaly when it is supported by only one detector. A record is produced only when at least two detection signals support the same date and metric. For example, a residual deviation may be supported by both the Z-score and univariate Isolation Forest detectors, or by a residual-based detector together with season comparison or the multivariate detector. This confirmation rule is used to reduce weak single-method findings and prevent one detector from independently determining the reference set. When several detectors identify the same date and metric, their outputs are consolidated into one candidate anomaly record and the supporting methods are stored with that record. This also makes the later expert-validation process more transparent because each candidate can be traced back to the methods that supported it.

Season comparison is used to handle contextual anomalies, especially during launch periods. In a launch regime, each target launch day is compared with the same launch-relative day from the immediately prior season. For example, launch day 0 is compared with launch day 0 of the previous season, day 1 with day 1, and so on. This matters because comparing a launch period only against calm historical periods would risk treating expected launch behavior as anomalous. However, the framework also needs a rule for deciding when a launch-period difference is large enough to be saved as a candidate anomaly. For this reason, simple per-metric deviation thresholds are calibrated from non-launch periods. Launch windows are excluded from this calibration, and for each metric the framework measures how far the observed value usually deviates from its expected value during calmer periods. The 90th percentile of these deviations is used as the medium threshold and the 95th percentile is used as the critical threshold. During launch periods, these thresholds are relaxed by a factor of 1.5 because launch weeks are expected to move more strongly than ordinary periods. These thresholds are used as a practical filtering step, not as the main object of evaluation. They help prevent the framework from treating every launch movement as anomalous while still allowing unusually large launch-period deviations to be saved as candidate records. In normal periods, season comparison is used mainly as supporting context rather than as a standalone record-generating detector, because simple period-over-period differences can reflect trend rather than anomaly.

Multivariate Isolation Forest is used to detect days that are unusual as combinations of metrics. This is important because game incidents can affect several KPIs at once, and the pattern across metrics may be more informative than any single metric in isolation. For example, a DAU movement may be interpreted differently if revenue, registrations, and matchmaking quality are all moving in related or contradictory directions. The multivariate detector is fitted on a residual matrix containing all monitored metrics, and when it flags a day, the framework attributes the finding to the top contributing metrics based on standardized residual deviation.

4.7 Changepoint and Drift Detection

Point-anomaly methods are not sufficient for all relevant KPI behavior. A game metric can shift into a new level after an update, technical change, or gameplay event without producing one extreme day. Similarly, a metric may drift gradually over several days or weeks, where no individual observation is unusual enough to trigger a point detector. To address these cases, the framework includes a changepoint layer in addition to the weekly point layer.

The changepoint layer runs on a longer time series and identifies structural changes dated at their onset. A changepoint is then attached to a target week only if its onset falls inside that week. This distinction is important because the changepoint layer is not asking whether a single day is far from expectation. It is asking whether the underlying behavior of the series has shifted. Therefore, changepoint records are treated as separate anomaly types from weekly point records.

PELT is used to detect regime shifts. In the framework, PELT runs on the z-scored trend component so that it focuses on structural changes in the level of the series rather than weekly noise. The ‘L2‘ model is used because the goal is to detect changes in mean level. The penalty controls how many changepoints are selected, while the minimum segment size of 7 prevents the detector from placing several changepoints inside the same week.

CUSUM is used to detect sustained drift. It runs on the z-scored deseasonalized series, which removes weekly movement before accumulating same-direction deviations. This allows the detector to identify persistent movement that may not be visible as a single-day anomaly. CUSUM is therefore useful for gradual deterioration or improvement in metrics such as registrations, paying users, or matchmaking quality.

The changepoint layer requires more history than the weekly point layer. In the framework, at least 90 days of data are required for changepoint analysis. This ensures that regime shifts and drift are evaluated over a meaningful historical period rather than over a short and potentially unstable sample. For each evaluation period, the changepoint layer uses the historical series available up to the end of the target period and then retains only changepoint onsets that fall within the target week. This differs from the point layer, which holds the target week out entirely and estimates baselines only from prior history. The target week is included here because a changepoint is defined by the contrast between the segments before and after the onset, so an onset can only be identified and dated once the days that follow it have been observed. This is suitable for offline ground-truth construction because the purpose is to identify and date structural changes as accurately as possible rather than to forecast them in real time.

The parameters used in the framework are fully specified through a fixed set of values covering decomposition, detector settings, changepoint settings, threshold logic, random seeds, and history requirements. These values are held constant across every target period so that differences between the candidate anomaly sets are caused by changes in the underlying KPI data rather than changes in the framework configuration. Fixed parameters also make the framework reproducible, because running it again on the same data produces the same expected values, detector outputs, and candidate anomaly records. Random seeds are therefore fixed for stochastic components such as Isolation Forest. The complete parameter set is listed in Appendix A.1. Together with the methodology above, these parameters define the preprocessing, baseline selection, thresholds, detector settings, and consolidation rules used to construct the statistical reference candidates.

4.8 Validation and Reference Finalization

After the framework produced the candidate anomaly records, the framework implementation and its outputs were reviewed in two stages. First, one Tencent analyst reviewed the framework code, detector selection, parameter settings, and overall methodological logic. This review was used to confirm that the implementation matched the methods described in the thesis and that the selected detectors and parameters were reasonable for the Delta Force KPI setting. The review covered the use of STL and rolling-median expected values, the residual-based point detectors, season comparison, multivariate detection, changepoint and drift methods, and the rule requiring support from more than one detector before a candidate anomaly was recorded.

The second stage focused on validating the candidate anomalies themselves. Three Tencent analysts and domain experts participated in this stage, including the analyst who had also reviewed the framework implementation. The evaluation weeks were divided approximately equally across the reviewers so that each reviewer was responsible for checking a separate subset of the candidate records. They reviewed every candidate anomaly assigned to them using internal presentations, dashboards, game reports, operational context, and their extensive prior knowledge of the subject. These sources made it possible to check whether a statistically unusual movement was also considered unusual by the teams that regularly work with the game data. This expert review was necessary because I did not have access to all of the internal reporting sources used by these teams and did not have the same level of accumulated domain knowledge required to independently interpret every candidate anomaly within its full context.

A candidate was retained when the reviewer concluded that the detected movement should be treated as an anomaly within the wider context. A candidate was removed when internal dashboards, reports, presentations, or known game events showed that the movement should not be considered anomalous. Every change made during validation was a removal of an existing candidate, as no reviewer added a new anomaly record. This suggests that the combined detectors provided broad coverage of the relevant anomalies and that expert review mainly helped remove findings that were statistically unusual but explainable in the wider game context.

The reviewers also reported that, within the periods assigned to them and based on the internal information available, the framework had not missed anomalies that should have been included. Approximately 90% of the original candidate records were therefore retained in the final reference set. This provides stronger support

for the coverage of the framework than a review that only confirms or rejects the records already presented. However, it should not be interpreted as a measured recall estimate, because no separate set of independently produced labels was created and each evaluation period was reviewed by one expert. After this process, the retained records became the verified reference set used in the LLM experiments.

The main cases that required additional judgment were periods close to a season launch. These periods are difficult because they are no longer part of the initial launch window, but the game has not fully returned to calm behavior either. During this post-launch decay phase, player activity, revenue, and registrations can remain elevated after a launch and then gradually decline towards a more stable level. The framework accounts for this context through season comparison and by checking whether movements are consistent with similar post-launch behavior from earlier seasons. However, post-launch decay is not always smooth, and some movements remain difficult to interpret from the statistical output alone. In these cases, the candidate records were checked against the surrounding trend, comparable season behavior, internal presentations, and known game context before being included in the verified reference set.

Metrics with very small values also required careful handling. Matchmaking failure rate is close to zero, so small absolute movements can produce large percentage deviations. The framework already includes denominator floors and metric-specific filtering to reduce this issue, but records for this metric were still reviewed carefully because the operational meaning of a small rate movement depends on the actual scale of the change. This was one of the main cases where access to internal dashboards and domain experience helped determine whether a statistically large deviation was also operationally meaningful.

Overall, the validation supported the selected framework design for the Delta Force KPI setting. STL-based residual detection was appropriate for metrics with clear weekly structure, while the lag-7 gate and rolling-median fallback avoided forcing seasonal decomposition on metrics where it was not supported by the data. Season comparison was important during launch and post-launch periods, and the changepoint and drift layer captured structural changes that daily point detectors did not express as clearly. The requirement that a candidate anomaly be supported by more than one detector also reduced weak single-method findings before the expert review took place. After the validation process, the retained records became the verified ground-truth set used in the LLM experiments.

4.9 Scope of the Reference Framework

Before moving to the experiments, it is important to clarify how the reference framework should be interpreted. The framework is designed as an appropriate and reproducible anomaly detection procedure for the Delta Force KPI setting. It combines methods from the related work with choices that are specific to the behavior of the data, such as weekly expected-value construction, launch and post-launch comparison, per-metric thresholds, and changepoint detection. The manual validation step then adds domain review so that the final reference set is not only statistically consistent, but also meaningful in the context of the game.

The framework is therefore not meant to be a universal or final anomaly detection algorithm for all live-service games. Its purpose in this thesis is to create a stable and defensible reference set against which LLM outputs can be evaluated and to see which methods work best in this context. This distinction matters because the later precision, recall, and F1 scores measure how closely the LLM-based analytics agent reproduces this verified reference set. They should be interpreted as measures of agreement with a well-defined and reproducible reference. The limitations of the framework and the remaining difficult cases are discussed further in Section 8.

5 Experimental Design

The methodology section explained how the verified reference set was created. With that reference in place, the remaining subquestions can be tested experimentally. The second subquestion asks whether an LLM-based analytics agent can reproduce the verified anomalies accurately. The third asks whether methodological guidance improves consistency and reliability. The fourth asks whether performance depends on the underlying model used inside LeoX. The experiments are designed around these questions. Experiment 1 compares guided and unguided anomaly detection accuracy. Experiment 2 tests whether guided runs are stable when the same task

is repeated. Experiment 3 compares different models under the same guided setup. All experiments use the verified reference set as ground truth. Each LLM run is given a target analysis period and the monitored Delta Force KPI data, and its output is converted into anomaly records. The main comparison is based on whether the LLM identifies the correct date, metric, and anomaly type. This keeps the evaluation focused on whether the agent found the same anomaly as the verified reference.

The LLM tasks are evaluated over one-week target periods rather than isolated single days. This choice is important for both practical and evaluation reasons. In Tencent, live-game KPI monitoring is usually reviewed over recent reporting periods, where analysts need to know whether anything unusual happened during the week/month rather than checking each day as a separate task. For the evaluation, weekly windows also create a larger set of possible anomaly records within each run. If the task were framed as one isolated day at a time, many runs would contain very few or no anomalies which would make precision, recall, and F1 less informative as many runs would be needed to get sufficient data. A one-week target period therefore gives the LLM enough context and enough potential anomaly records for scoring, while also still keeping each run narrow enough for manual validation and comparison. The selected periods include both launch and non-launch regimes so that the experiments test the agent under stable monitoring conditions as well as during high-activity update periods.

5.1 Scoring Method

Throughout all experiments, the same matching logic is used to compare LLM outputs against the ground truths. Each LLM output is first converted into a set of structured anomaly records. These records are then matched against the reference using the tuple $(date, metric, anomaly\_type)$. This tuple is used because it captures when the anomaly happened, which KPI it affected, and what kind of anomaly it was. If any of these fields is wrong, the LLM has missed the anomaly. A detected anomaly is counted as a true positive when it matches a ground truth anomaly on these three fields. It is counted as a false positive when the LLM reports an anomaly that is not in the reference set, and as a false negative when a verified reference anomaly is missed. Precision, recall, and F1 are then calculated from these counts. This makes the evaluation more systematic than a qualitative review because each output is judged against the same matching rule.

The matching rule is strict enough to test whether the LLM identified the correct anomaly, but not so strict that it requires the LLM to reproduce every implementation detail of the reference framework. For example, the LLM may estimate a slightly different expected value or describe the magnitude differently while still identifying the same anomaly on the same date, metric, and anomaly type. For this reason, observed value agreement, expected value agreement, direction, and magnitude are treated as supporting diagnostics rather than as the main matching criteria. They are useful for understanding the quality of matched findings, but the main accuracy scores are based on whether the correct anomaly record was found.

The verified reference set was also finalized before the LLM experiments were scored and was applied consistently across all models and conditions. Every returned anomaly record was evaluated using the same matching rule based on date, metric, and anomaly type. Manual inspection was used only to ensure that the LLM outputs were interpreted and converted correctly into the required structured format. The purpose of this evaluation is to measure how accurately an LLM-based analytics agent can carry out the end-to-end anomaly-detection task and reproduce the verified findings, rather than to reassess the validity of the reference set.

5.2 Experiment 1: Guided versus Unguided Detection

The first experiment evaluates whether an LLM-based analytics agent can detect anomalies without detailed methodological guidance, and whether adding structured context improves anomaly detection accuracy. This experiment directly addresses the second subquestion, which asks how accurately an LLM-based analytics agent can detect anomalies compared to the ground truths. It also contributes to the third subquestion because it tests whether structured guidance helps the agent make better choices, such as selecting suitable baselines, accounting for weekly behavior, handling launch periods, and returning outputs in a consistent structure.

This is the foundational experiment because the later reliability and model benchmark experiments are only meaningful if guided anomaly detection first improves over the unguided baseline. This experiment uses the

same model, same provided data, same ground truths, same target periods, and same scoring method in both conditions. The only difference is whether the LLM has access to the anomaly detection Skill.

In the unguided condition, the LLM is given the Delta Force KPI data, the target period, the metrics to examine, and the required output format, but it is not given the anomaly-detection Skill. It must decide for itself how to approach the task, including which baselines to construct, which methods to use, and what should count as anomalous. In the guided condition, the LLM receives the same data and task, but also has access to the Skill described in the next subsection. This means that both conditions work from the same input data, while only the guided condition receives structured methodological and game-specific context.

Experiment 1 is evaluated across the verified periods listed in Table 1. These periods cover launch, post-launch, normal, and late-season regimes. This is important because a method that only works during calm weeks is not sufficient for live-game monitoring. Launch periods are more difficult because many KPIs move at the same time, while non-launch periods test whether the agent can identify more subtle deviations against recent trend and weekly behavior. The table also reports N_{GT}, the number of verified anomaly records for each week. This is included because the evaluation is performed at the anomaly-record level rather than only at the week level. Across the nine weeks, Experiment 1 evaluates *146* verified anomaly records, so the comparison is not based only on a small number of weekly outcomes but has a substantial amount of data behind it. One week contains no verified anomalies and is useful for checking whether the agent over-reports anomalies when the reference set is empty. The main outcome is whether an LLM-based analytics agent can carry out the anomaly detection task end-to-end, and how much game related methodological guidance improves its performance. This is measured by comparing the guided and unguided conditions in terms of precision, recall, and F1 against the same verified reference set.

Table 1: Evaluation periods used in Experiment 1

Start date	**End date**	**Period/context**	N_{GT}
2026-01-26	2026-02-01	Pre-S8 baseline week	12
2026-02-03	2026-02-09	S8 launch	32
2026-02-10	2026-02-16	Direct post-S8 week	26
2026-02-23	2026-03-01	Post-Launch Decay / no-anomaly control	0
2026-03-02	2026-03-08	Mid-season week	3
2026-03-16	2026-03-22	Mid-season week	4
2026-04-06	2026-04-12	Late-season week before S9	15
2026-04-21	2026-04-27	S9 launch	30
2026-04-28	2026-05-04	Direct post-S9 week	3
2026-05-05	2026-05-11	Post-Launch Decay week	21

5.2.1 Methodological Guidance through the Skill

The guided condition uses an anomaly detection Skill inside LeoX. A Skill is a reusable instruction layer that gives an LLM additional context and workflow guidance for a specific task. In this experiment, the Skill provides theoretical, methodological, and game-specific context rather than an executable anomaly detection procedure. It explains the main anomaly types that can occur in time-series data, why temporal baselines and seasonality matter, how season launches and post-launch periods affect KPI behavior, and which general detection methods may be useful for different anomaly patterns. It also explains the monitored KPI definitions, the data requirements for the analysis, and the importance of keeping the target period separate from the historical baseline.

The Skill does not contain any code or step-by-step instructions used to construct the final anomaly labels. It does not tell the model which dates or metrics should be flagged, and it does not provide the expected outputs for any evaluation period. The agent remains responsible for inspecting the provided data, deciding how to apply the available methodological guidance, implementing and executing its own analysis, interpreting the results, and returning the final anomaly records. The Skill is intended to act as a domain and methodological expert layer that helps the agent understand the task and make more informed analytical decisions instead of being a fixed procedure that determines how the analysis must be carried out. A summary of the context included in and excluded from the Skill is provided in Appendix A.2.

In the unguided condition, the model was provided with the attached KPI dataset, the target analysis period, the metrics to examine, and the required JSON reporting format. It was then asked to identify anomalies within the specified period using the available data. However, it did not have access to the anomaly-detection Skill and therefore received no additional task-specific guidance on metric interpretation, season or update context, temporal baselines, recurring patterns, deviation floors, suitable detection methods, or the interpretation of different anomaly types. The model therefore had to determine its own analytical approach using the provided data and its general knowledge.

In the guided condition, the model received the same data and core task together with the additional theoretical, methodological, and game-specific context contained in the Skill. This guidance explained relevant anomaly-detection concepts, the importance of temporal context, the possible impact of season launches and post-launch behavior, and the types of methods that may be useful for different anomaly patterns. However, it did not provide exact baseline calculations, fixed parameter values, step-by-step implementation instructions, code, or verified anomaly labels. The agent still had to decide how to apply the guidance, implement and execute its own analysis, and determine which records should be reported. The difference between the two conditions was therefore access to structured expert context rather than access to a predefined analytical procedure or the expected results. Representative reconstructions of the guided and unguided prompts are provided in Appendix A.3.

For evaluation purposes, both conditions were instructed to return the detected anomalies in a standardized JSON format. This made the outputs easy to download, convert into structured anomaly records, and compare consistently against the verified reference set. A simplified example of the required reporting structure is provided in Appendix A.4.

Overall, the purpose of this setup is to test whether an LLM-based analytics agent is capable of carrying out the anomaly detection task independently, and whether access to structured methodological and domain context improves the quality of its results.

5.3 Experiment 2: Run-to-Run Reliability

The second experiment evaluates the reliability of guided anomaly detection across repeated runs. This is different from accuracy because an agent can achieve a strong score in one run but still be unreliable if it produces different anomaly sets when the same task is repeated. For a monitoring workflow, consistency matters because users need to know whether a single guided run is likely to produce a dependable result or whether its output changes substantially across repeated executions.

To isolate this question, Experiment 2 keeps the model, Skill, prompt, input data, target week, and verified reference set fixed. Every run uses the exact same prompt and the same version of the anomaly detection Skill, and each run is started in a fresh LeoX session so that earlier outputs and conversation history do not carry over. The only difference between the runs is therefore the independent model execution. The model used in this experiment is Claude Opus 4.8 with the anomaly detection Skill. As explained previously, the Skill provides theoretical, methodological, and game-specific context but does not contain code, a step-by-step procedure, or any of the verified anomaly labels. The model must still decide how to apply this context and carry out the analysis independently in every run.

Five runs are performed for each selected week. This number was chosen as a practical balance between repeating the task enough times to observe whether meaningful run-to-run variation occurs and keeping the number of LeoX executions manageable within the scope of the thesis. The purpose is not to estimate every possible output the model could produce, but to examine whether the guided workflow remains reasonably consistent when the same task is repeated under identical conditions.

The selected weeks are shown in Table 2. They represent different levels and types of difficulty within the season cycle. The S8 launch week contains large movements across several KPIs, the direct post-S8 week represents a more difficult period where launch effects and post-launch behavior overlap, and the late-season week before S9 represents a more stable comparison period. This allows the experiment to examine whether

run-to-run variation differs depending on the context and difficulty of the target week.

Table 2: Evaluation periods used in Experiment 2

Start date	End date	Context	N_{GT}
2026-02-03	2026-02-09	S8 launch	32
2026-02-10	2026-02-16	Direct post-S8 week	26
2026-04-06	2026-04-12	Late-season week before S9	15

The outputs from all runs are converted into the same structured anomaly-record format and evaluated using the same matching rule as Experiment 1. Reliability is measured in two main ways. First, the experiment examines whether the accuracy scores remain stable across repeated runs. Precision, recall, and F1 are calculated separately for each run and then summarized for each week using the mean, standard deviation, coefficient of variation, and minimum/maximum range. These measures show whether the overall performance of the agent remains similar or changes meaningfully when the same analysis is repeated.

Second, the experiment examines whether the actual anomaly records returned by the agent are consistent across runs. This is necessary because two runs can achieve similar precision, recall, and F1 scores while identifying different combinations of anomalies. The records from each run are represented using the same matching key as the other experiments, based on date, metric, and anomaly type. Pairwise Jaccard similarity is then calculated between the anomaly sets from every pair of valid runs, and the average of these pairwise comparisons is used to summarize record-level consistency for each week. A higher value means that repeated runs returned more similar anomaly sets, while a lower value indicates greater disagreement between runs.

The experiment also examines detection frequency at the individual-record level. For every verified reference anomaly, the analysis counts how many of the five runs detected the record. This shows which anomalies are consistently identified and which ones are only found in some runs. False-positive frequency is calculated in the same way by counting how often each non-reference record is reported across the five runs. This makes it possible to distinguish between occasional run-specific errors and false positives that repeatedly appear for the same date, metric, and anomaly type.

Together, these measures evaluate reliability from both a performance and an output perspective. The score summaries show whether the overall level of accuracy remains stable, while Jaccard similarity and record-level detection frequency show whether the model is consistently identifying the same anomalies.

5.4 Experiment 3: Model Benchmark

The third experiment evaluates whether guided anomaly detection performance differs depending on the underlying LLM used inside LeoX. The experiment keeps the provided data, target periods, anomaly detection Skill, prompt structure, verified reference set, and scoring method fixed, while changing only the model used to carry out the analysis. The purpose is to examine whether the guided workflow produces similar results across different models or whether some models are better able to interpret the guidance, implement the required analysis, and identify the verified anomaly records.

This experiment is designed as a small-scale descriptive benchmark rather than as a definitive ranking of the models. The aim is to observe how different LLMs approach the same guided analytics task under comparable conditions. Each model must inspect the data, decide how to apply the theoretical and game-specific context provided through the Skill, execute its own analysis, and return the detected anomalies. As explained previously, the Skill does not contain code, step-by-step implementation instructions, or the verified anomaly labels. Therefore, differences in performance can still arise from how each model interprets the task, selects and implements methods, reasons about the game context, and structures its final output.

The benchmark compares three models available through LeoX: Claude Opus 4.8, DeepSeek-V4-Pro, and Tencent's deployment of GLM-5.2. These models were selected because they represent different model families available within LeoX and provide a useful comparison across the supported model deployments. Claude Opus 4.8 is included as the main Anthropic model used in the earlier experiments and therefore provides a direct

comparison point for the benchmark. Anthropic positions its Opus models for complex reasoning, coding, and agentic tasks, which makes the model relevant for an analytics workflow that requires the independent implementation and execution of an anomaly detection procedure [16]. DeepSeek-V4-Pro is included as a separate mixture-of-experts model family designed for reasoning, coding, long-context processing, and also agent-style tasks [17]. Its inclusion makes it possible to examine whether a model with a different architecture and training background can use the same Skill and provided data as effectively as Claude. The third model is Tencent's GLM-5.2 deployment. This deployment is based on the GLM model family and uses FP8 quantization within the internal environment. Quantization reduces the numerical precision used to represent parts of the model, which can reduce memory and computational requirements during inference. GLM models are designed with a focus on reasoning, coding, long-context processing, and agentic workflows, making this model another relevant candidate for the task [18]. Since the benchmark evaluates the deployments available inside LeoX, the results should be interpreted as performance within this environment rather than as a general comparison of the complete public model families.

The selected evaluation periods are shown in Table 3. They include two launch weeks, one difficult direct post-launch week, one direct post-S9 week, and one late-season week. Together, the five periods contain 106 verified anomaly records and allow the models to be compared across different types of KPI behavior rather than only during one part of the season cycle.

Launch weeks test whether the models can handle periods where several KPIs move strongly at the same time. The direct post-launch weeks are included because these periods require more contextual interpretation, as KPI levels may still be affected by the launch while beginning to move towards a more stable level. The late-season week provides a comparison under more stable conditions.

Each model is run once for each of the five selected weeks, resulting in $3 \times 5 = 15$ model evaluations. Existing Claude Opus 4.8 guided runs are reused where the model, prompt, Skill, data, and experimental conditions match those required for the benchmark. This avoids repeating identical runs unnecessarily while preserving comparability between the evaluated conditions.

Table 3: Evaluation periods used in Experiment 3

Start date	**End date**	**Period/context**	N_{GT}	**Reason for inclusion**
2026-02-03	2026-02-09	S8 launch	32	Launch week with a large reference set
2026-02-10	2026-02-16	Direct post-S8 week	26	Difficult post-launch period
2026-04-06	2026-04-12	Late-season week before S9	15	More stable pre-launch comparison
2026-04-21	2026-04-27	S9 launch	30	Second launch week for comparison
2026-04-28	2026-05-04	Direct post-S9 week	3	Post-launch comparison after S9

Each model is evaluated using the same record-level matching logic based on date, metric, and anomaly type. True positives, false positives, and false negatives are calculated for each week and used to produce precision, recall, and F1. Performance is then summarised across the five evaluated weeks so that the models can be compared using the same scoring procedure. Output validity is also recorded across all 15 runs. A run is considered valid when the model completes the task and returns anomaly records in the required JSON structure so that they can be processed by the evaluation code.

The results are interpreted descriptively because the purpose of this experiment is to compare how the available models behave on the selected guided task and not to establish a universal ranking of LLM performance. Experiment 2 examines whether the guided setup produces reasonably consistent results across repeated Claude Opus 4.8 runs, which provides useful context for interpreting the benchmark. However, each model-week combination in Experiment 3 is evaluated once, so small differences between models should not be treated as conclusive. The main goal is therefore to identify whether meaningful performance differences appear within this specific workflow and which of the evaluated LeoX models produces the most dependable combination of anomaly-detection accuracy, false-positive control, and valid structured output.

6 Experimental Results

This section presents the results of the three experiments described above. The results are organized in the same order as the experimental design. First, the guided and unguided conditions are compared to evaluate whether an LLM-based analytics agent can carry out the anomaly detection task end-to-end, and whether methodological context improves its accuracy. This experiment is therefore not only a Skill comparison, but also a test of whether the agent can inspect the provided KPI data, construct baselines, identify anomaly records, and return outputs that can be evaluated against the verified reference set. Second, the repeated-run experiment assesses whether guided outputs remain stable when the same task is run multiple times. Third, the model benchmark compares whether performance differs across the LLMs available through LeoX. Across all experiments, the results are evaluated against the same verified reference set using record-level matching based on date, metric, and anomaly type. Weeks with no verified anomalies are treated separately as false-positive controls because recall and F1 are not meaningful when the reference set is empty.

6.1 Experiment 1: End-to-End Detection and Guided vs Unguided Accuracy

Experiment 1 compares guided and unguided anomaly detection using Claude Opus 4.8. In both conditions, the model is provided with the relevant KPI data, target analysis week, metrics to examine, and the information that the dataset represents live-game data. The unguided condition receives no additional task-specific guidance on anomaly detection or how the game context may affect KPI behavior. The guided condition receives the same core task together with the theoretical, methodological, and game-specific context contained in the anomaly detection Skill. Outputs are matched against the verified reference set using the tuple $(date, metric, anomaly\_type)$. Nine target weeks contain 146 verified anomaly records and are evaluated using precision, recall, and F1. One additional week contains no verified anomalies and is treated separately as a false-positive control.

6.1.1 Overall Accuracy

After running both conditions on the same target weeks, Table 4 shows the per-week results. The table reports precision, recall, and F1 for both conditions together with the change in F1. When a model returns no anomaly records for a week containing verified anomalies, precision is undefined and is shown using a dash. Recall is 0 because none of the verified anomalies were detected, and F1 is treated as 0 for the purpose of comparison.

Table 4: Experiment 1 per-week results for guided and unguided anomaly detection

Week	Period/context	N_{GT}	Guided P	Guided R	Guided F1	Unguided P	Unguided R	Unguided F1	ΔF1
2026-01-26	Pre-S8 baseline week	12	0.571	0.333	0.421	0.227	0.417	0.294	0.127
2026-02-03	S8 launch	32	0.839	0.813	0.825	0.700	0.844	0.765	0.060
2026-02-10	Direct post-S8 week	26	0.839	1.000	0.912	0.429	0.192	0.266	0.647
2026-03-02	Mid-season week	3	1.000	0.333	0.500	1.000	0.667	0.800	-0.300
2026-03-16	Mid-season week	4	1.000	1.000	1.000	–	0.000	0.000	1.000
2026-04-06	Late-season week before S9	15	1.000	1.000	1.000	1.000	0.333	0.500	0.500
2026-04-21	S9 launch	30	0.966	0.933	0.949	0.571	0.467	0.514	0.435
2026-04-28	Direct post-S9 week	3	0.750	1.000	0.857	0.100	0.667	0.174	0.683
2026-05-05	Post-S9 stabilization week	21	1.000	1.000	1.000	–	0.000	0.000	1.000
2026-02-23	No-anomaly control week	0	–	–	–	–	–	–	–

The per-week results show that the guided condition performs better in eight of the nine scored weeks. However, the weekly scores should be interpreted together with the number of ground-truth anomalies in each period. Some weeks contain a large number of verified records, while others contain only a few. For example, the S8 launch week contains 32 ground-truth anomalies, while the weeks beginning on 2026-03-02 and 2026-04-28 each contain only 3. In these smaller weeks, detecting or missing just one record can cause a large change in precision, recall, and F1. This explains why the unguided condition achieves a higher F1 on 2026-03-02 even though the result is based on only three ground-truth anomalies. The weekly scores are therefore useful for showing where performance changes across different periods, but the aggregate results provide a more reliable overall comparison between the two conditions.

Table 5 reports the aggregate results across the nine scored weeks. Micro-averaged precision, recall, and F1 are calculated by combining the true positives, false positives, and false negatives across all evaluated anomaly records before calculating the final metrics. Macro F1 is calculated by averaging the weekly F1 scores, which gives each week equal weight regardless of the number of verified anomalies it contains.

Table 5: Experiment 1 aggregate results across the nine scored weeks

Condition	TP	FP	FN	Micro P	Micro R	Micro F1	Macro F1
Guided	128	15	18	0.895	0.877	0.886	0.829 ± 0.219
Unguided	60	57	86	0.513	0.411	0.456	0.368 ± 0.297

The aggregate results show a big improvement when the anomaly detection Skill is provided. Across the nine scored weeks, the guided condition correctly identifies 128 of the 146 ground-truth anomaly records, while the unguided condition identifies 60. As a result, the guided condition misses only 18 verified anomalies compared with 86 in the unguided condition, increasing recall from 0.411 to 0.877. The guided condition also produces fewer incorrect anomaly records, with 15 false positives compared with 57 in the unguided condition. This increases precision from 0.513 to 0.895. When precision and recall are combined, micro F1 increases from 0.456 in the unguided condition to 0.886 in the guided condition.

Micro F1 is treated as the main aggregate result because it evaluates performance across all 146 ground-truth anomaly records together. This gives more weight to weeks containing larger reference sets and prevents weeks with only a few anomalies from having too much influence on the overall conclusion. Macro F1 is included as a secondary measure because it gives every week equal weight, regardless of how many ground-truth anomalies it contains. The guided condition reaches a macro F1 of 0.829 ± 0.219, compared with 0.368 ± 0.297 for the unguided condition. This supports the same overall conclusion, while the lower standard deviation shows that guided performance also varies less across the selected weeks. Run-to-run reliability is examined separately in Experiment 2.

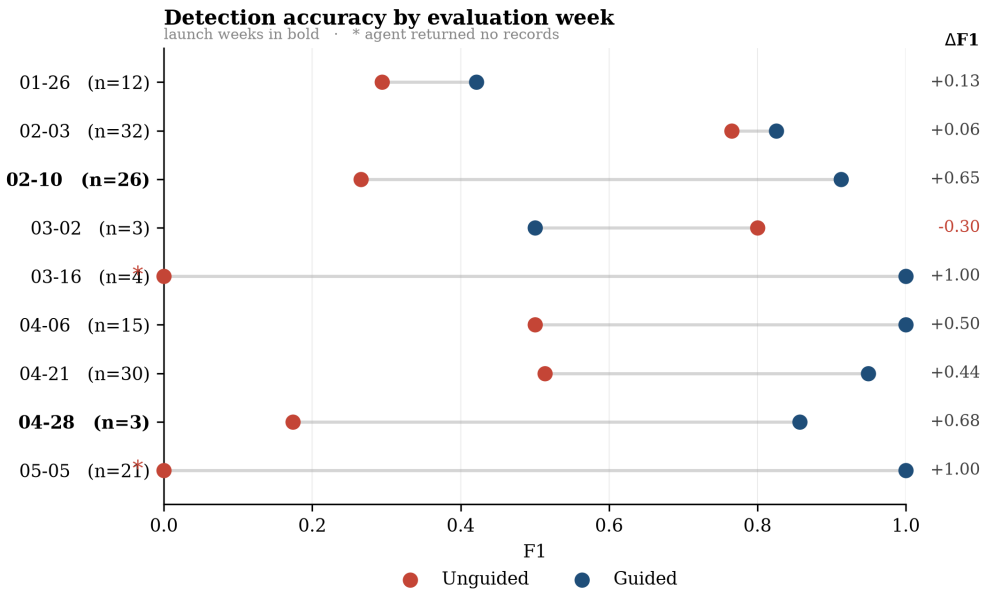

Figure 5: Guided and unguided F1 by evaluation week.

Figure 5 shows that the improvement from guidance is not driven by one target week. The guided condition performs better in most scored periods, with especially large gains on 2026-02-10, 2026-04-28, and 2026-05-05. These weeks occur after season launches, when KPI behavior is more difficult to interpret because the metrics may remain affected by the launch while also beginning to move towards a more stable level. The only week

where the unguided condition achieves a higher F1 is 2026-03-02, which contains three verified anomaly records.

6.1.2 Where Guidance Improves Performance

To examine where the Skill provides the most value, the scored weeks are grouped according to their position in the season cycle. Figure 6 summarizes this comparison. The launch group contains the S8 and S9 launch weeks. The post-launch group contains the periods immediately after or shortly after a season launch. The calm group contains the remaining mid-season and late-season weeks.

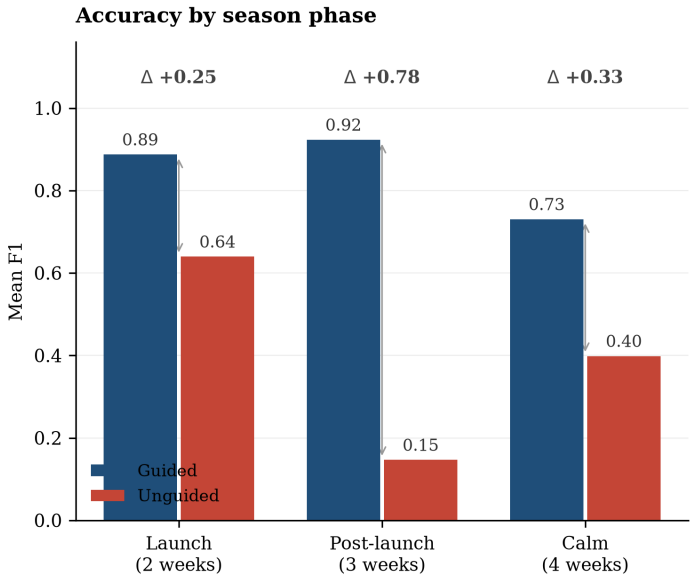

Figure 6: Mean F1 by season phase.

The unguided condition performs best during launch weeks, where it reaches a mean F1 of 0.640. This is likely because launch periods contain large and visible movements across activity, acquisition, and monetization KPIs. The most obvious anomaly records can therefore be identified without extensive game-cycle context. Guidance still improves performance during launch weeks, but the difference is smaller than in the post-launch group.

The largest difference appears during post-launch periods. The guided condition reaches a mean F1 of 0.923, while the unguided condition reaches 0.146. These periods are difficult because KPI levels may remain elevated after a launch while also beginning to decline from the launch peak. A movement can appear unusual relative to recent values while still being expected for that point in the season cycle. The Skill provides theoretical and game-specific context about launches, post-launch behavior, temporal comparisons, and the importance of selecting an appropriate analytical baseline. The results suggest that this context helps the model make more suitable analytical decisions during post-launch periods.

The unguided outputs also vary substantially across the three post-launch weeks. On 2026-02-10, the model reports only 7 records against 26 verified anomalies, resulting in severe under-detection. On 2026-04-28, it reports 20 records against only 3 verified anomalies, resulting in substantial over-detection. On 2026-05-05, it returns no anomaly records despite 21 verified anomalies being present. The unguided condition therefore under-reports, over-reports, and returns no findings within periods that share a similar position after a season launch. This indicates that the problem is not simply one consistently high or low detection threshold. Instead, the model does not reliably adjust its analysis to the context of each post-launch period when this context is not provided through the Skill.

6.1.3 Interpretation Quality of Matched Anomalies Example

To provide a more detailed example of why the guided and unguided accuracy scores differ, the matched records from the S9 launch week were examined beyond the main precision, recall, and F1 measures. This week was selected as a representative launch period because both conditions detected several anomalies, which made it possible to compare how the same KPI movements were interpreted after a record had been matched.

The guided condition detected 28 of the 30 ground-truth anomalies, compared with 14 in the unguided condition. Among the guided true positives, direction and anomaly type agreed with the reference for every record, magnitude agreed for 20 of the 28 records, and the expected value agreed for 27. Expected-value agreement was defined using the same tolerance applied in the scoring code, where a value was treated as agreeing when its relative difference from the reference expected value was within ±5%. In comparison, among the 14 unguided true positives, anomaly type agreed for 2 records, direction for 4, magnitude for 7, and the expected value for only 1. The median percentage difference between the unguided expected values and the reference expected values was 51.3%, compared with 0.0% for the guided condition.

This example shows that the difference between the two conditions is not only caused by one model being more or less sensitive when deciding whether to report an anomaly. Even when the unguided model identifies the correct date and metric, it can construct a different expectation for the KPI and therefore interpret the movement differently from the verified reference. The guided condition is therefore more accurate not only in the number of records it detects, but also in how it interprets the anomalies that it identifies.

6.1.4 No-Anomaly Control Week

The week from 2026-02-23 to 2026-03-01 contains no verified anomaly records and is therefore evaluated separately from the nine scored weeks. Precision, recall, and F1 are not informative when both the verified and predicted anomaly sets are empty, so the relevant outcome is whether either condition reports false positives.

Both the guided and unguided conditions return no anomaly records for this week. This shows that neither condition reports false positives during the selected control period. It also shows that the improvement in the guided condition is not produced by reporting more anomalies in every target week. The control result should still be interpreted together with the wider results, since the unguided condition also returns no records during 2026-05-05 despite that week containing 21 verified anomalies.

Overall, Experiment 1 shows that Claude Opus 4.8 is capable of carrying out the anomaly detection task using the provided KPI data, but its unguided performance varies substantially across the evaluated periods. Structured methodological and game-specific context improves record-level agreement with the verified reference set and leads to more accurate interpretation of the matched anomalies. The strongest improvement appears during post-launch periods, where the wider game context is especially important for interpreting KPI movement. Whether the guided performance remains consistent across repeated runs is evaluated separately in Experiment 2.

6.2 Experiment 2: Run-to-Run Reliability

Experiment 2 evaluates whether the guided Claude Opus 4.8 configuration produces similar results when the same anomaly-detection task is repeated under identical conditions. Five independent runs were completed for each of the three selected weeks. All 15 executions returned valid JSON outputs that could be processed using the standard evaluation procedure. The results therefore reflect differences in the anomaly records produced by the model rather than failures to complete the task or follow the required output format.

6.2.1 Overall Performance Stability

Table 6 summarizes the performance of the five repeated runs for each target week. The late-season week beginning on 2026-04-06 was perfectly stable as every run identified all 15 reference anomalies without producing any false positives, resulting in an F1 of 1.000 in all five runs. Performance also remained high during the direct post-S8 week, with a mean F1 of 0.967 and a standard deviation of 0.045. The S8 launch week was more

variable, although its mean F1 remained 0.883. Its standard deviation of 0.072 was the largest of the three evaluated periods.

Table 6: Run-to-run reliability across the three Experiment 2 periods

Week	N_{ref}	Mean P	Mean R	Mean F1	F1 SD	Mean Jaccard
2026-04-06	15	1.000	1.000	1.000	0.000	1.000
2026-02-10	26	0.952	0.985	0.967	0.045	0.881
2026-02-03	32	0.961	0.819	0.883	0.072	0.761

The results show that repeated guided runs were most stable in the late-season period, while greater variation appeared during the direct post-launch and S8 launch weeks. Despite this variation, mean F1 remained high in both periods, reaching 0.967 for the direct post-S8 week and 0.883 for the S8 launch week. The difference between the stability of the overall scores and the consistency of the underlying anomaly records is examined in the following subsections.

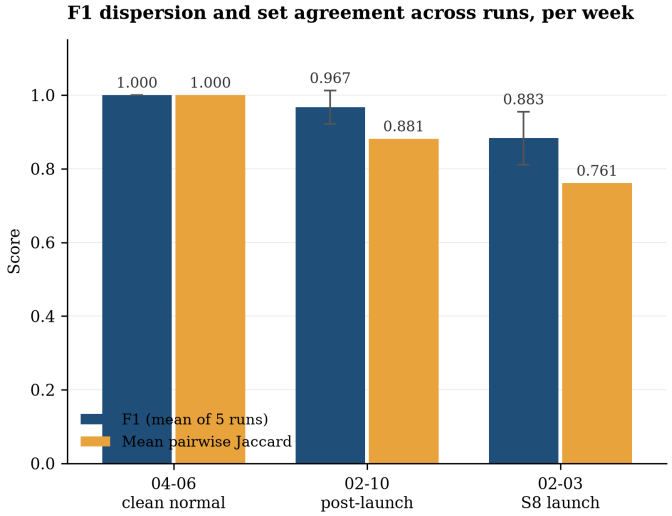

Figure 7: Mean F1 and mean pairwise Jaccard similarity across repeated runs.

6.2.2 Record-Level Consistency

The F1 results indicate that the overall level of performance was relatively stable, but they do not show whether each run returned the same anomaly records. This distinction is particularly relevant for the S8 launch week, where mean precision remained high at 0.961 while mean recall was lower at 0.819. The variation during this period was therefore driven mainly by differences in which verified anomalies were recovered, rather than by a large number of incorrect additional records. Figure 7 compares mean F1 with mean pairwise Jaccard similarity to examine this record-level consistency.

This distinction is visible in the direct post-S8 results. Three runs reproduced all 26 reference records without false positives. Another run also returned exactly 26 records, but only 24 matched the reference set, while two were false positives. A fifth run recovered all 26 reference records but also returned five additional false-positive records. The number of returned records could therefore be identical or close to the size of the reference set even when the membership and accuracy of the output differed. This shows why record-level similarity provides information that cannot be obtained from output volume or aggregate F1 alone.

In addition to comparing the final anomaly-record sets, the analysis examined whether repeated runs reported similar overall detection methods. For each run, the methods mentioned across all returned records were combined into one run-level method set. Pairwise Jaccard similarity was then calculated between these run-level sets. The mean method-set Jaccard similarity was 0.933 for the direct post-S8 week and 1.000 for the other two periods. This indicates that repeated runs generally reported the same broad families of detection methods. However, this comparison was performed at the run level rather than for each individual anomaly, and it did not

compare the reported methods directly with those supporting the verified reference records.

Follow-up prompts were used to request additional information about the parameters, thresholds, baseline windows, preprocessing choices, and intermediate detector results used in each run. However, the resulting explanations were often incomplete or inconsistent, and the model sometimes reported different analytical choices when asked again about the same run. It was therefore not possible to determine whether methods with the same reported name were implemented identically across executions or whether they were applied to the same individual records. This limited the extent to which differences between runs could be traced back to specific methodological decisions rather than only to the final anomaly records. The issue is discussed further in the limitations section.

6.2.3 Detection Frequency of Individual Reference Anomalies

Figure 8 shows how often each verified anomaly was recovered across the five runs. All 15 reference anomalies from the late-season week were found in every run. During the direct post-S8 week, 24 of 26 records were detected in all five runs and the remaining two were detected in four runs. The S8 launch week contained substantially more record-level variation as 20 of 32 reference anomalies were found in every run, 11 were found intermittently, and one was not recovered in any run. Figure 8 shows this variation.

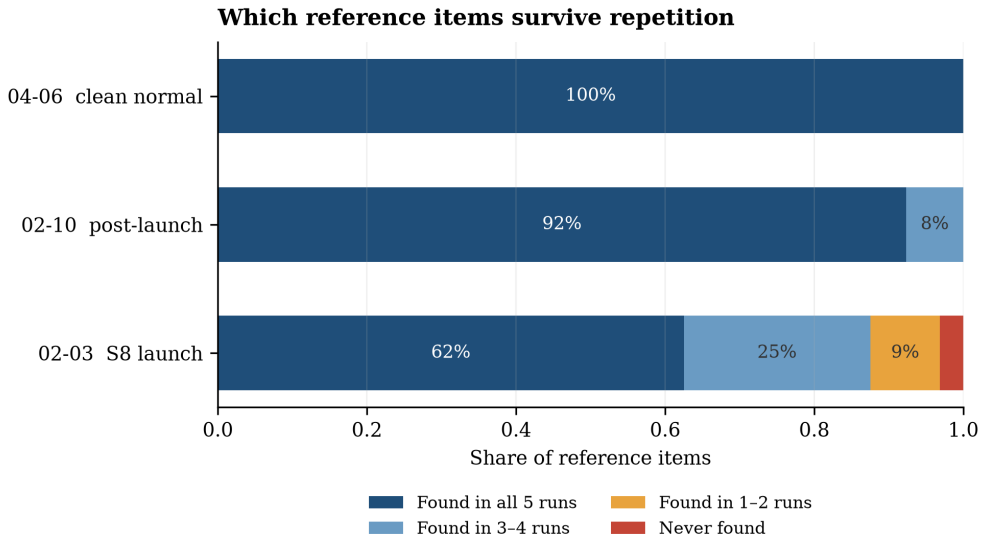

Figure 8: Distribution of verified reference anomalies according to the number of repeated runs in which they were detected. Percentages are calculated separately for each evaluation week.

Across the three periods, 59 of the 73 verified anomalies (80.8%) were detected in every run, 13 were detected intermittently, and 1 was not detected in any run. The consistently recovered records included every verified DAU and new-registrations anomaly across the three evaluated weeks. This indicates that the model was highly reproducible for these metrics under the guided configuration. In contrast, the less stable records were concentrated mainly in the S8 launch week. Of the 12 launch-week anomalies that were not detected in all five runs, six were matchmaking-success-rate point anomalies. Several daily-revenue and paying-users point anomalies were also recovered in only two or three runs. The only reference record that was missed by all five runs was the matchmaking-failure-rate point anomaly on 2026-02-03.

Most of the false-positive behavior was concentrated in matchmaking failure rate. This metric accounted for most of the post-launch false positives and for the more frequently repeated false positive during the S8 launch week. This may be partly explained by its very low baseline, where small absolute movements can appear large in relative terms. The effect of this metric's small scale is discussed further in the limitations section.

Overall, Experiment 2 shows that the guided Claude Opus 4.8 configuration produced valid and generally

high-performing outputs across all repeated runs. Reliability was strongest during the late-season period and slightly lower during the launch period, where the model did not always recover the exact same subset of reference anomalies. However, the overall run-to-run variation remained small across the evaluated periods. These results support using a single guided run as an informative analysis, while also showing that exact record-level reproduction may still vary slightly in more difficult cases.

6.3 Experiment 3: Guided Performance Across Models

Experiment 3 compares the performance of three LLM models under the same guided anomaly-detection setup. Claude Opus 4.8, DeepSeek-V4-Pro, and GLM-5.2 were provided with the same anomaly-detection Skill, prompt structure, KPI data, target periods, and verified reference set. Their outputs were evaluated using the same record-level matching rule based on date, metric, and anomaly type. Each model was evaluated once on each of the five included weeks, resulting in 15 model-week evaluations. All 15 outputs were returned in a valid structure and could be processed using the same evaluation procedure. Experiment 2 showed that the guided setup produced relatively small run-to-run variation overall, which supports the use of single-run model comparisons as an informative benchmark. However, because each model-week combination was evaluated once, the results are still interpreted descriptively rather than as a definitive ranking of the underlying model families. This is important because run-to-run variation was only tested for Claude Opus 4.8 and remains unknown for the other models.

6.3.1 Aggregate Model Performance

Table 7 reports the aggregate performance across the five evaluation weeks. Micro-averaged metrics are calculated after pooling the true positives, false positives, and false negatives from all weeks. Macro F1 gives each week equal weight by averaging the five weekly F1 scores.

Table 7: Aggregate Experiment 3 results across the five evaluated weeks

Model	TP	FP	FN	Micro P	Micro R	Micro F1	Macro F1
Claude Opus 4.8	98	12	8	0.891	0.925	0.907	0.909
DeepSeek-V4-Pro	74	37	32	0.667	0.698	0.682	0.585
GLM-5.2	84	43	22	0.661	0.792	0.721	0.628

Claude achieved the highest aggregate performance, with a micro F1 of 0.907 and a macro F1 of 0.909. GLM reached a micro F1 of 0.721 and a macro F1 of 0.628, while DeepSeek reached 0.682 and 0.585, respectively. The ordering was therefore the same under both aggregation methods. This indicates that Claude's stronger aggregate result was not produced only by the larger reference weeks receiving more weight in the micro-average.

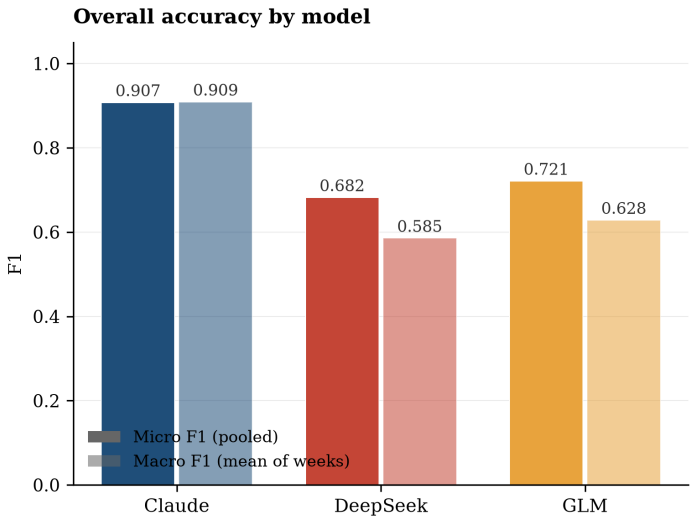

Figure 9: Micro and macro F1 across the five evaluated weeks for each model.

Claude showed a clear aggregate advantage over the other two models under both measures. By comparison, the difference between GLM and DeepSeek was relatively small, with a micro F1 difference of 0.039 and a macro F1 difference of 0.043. Given that each model-week combination was evaluated only once, the results do not establish that GLM would consistently outperform DeepSeek across repeated executions. The more defensible conclusion is that Claude performed clearly better in the given context, while GLM and DeepSeek produced broadly comparable aggregate results.

6.3.2 Performance Across Evaluation Weeks

Table 8 shows how the model results varied across the individual evaluation periods.

Table 8: F1 by evaluation week and model

Week	Period/context	N_{ref}	Claude	DeepSeek	GLM
2026-02-03	S8 launch	32	0.825	0.691	0.844
2026-02-10	Direct post-S8 week	26	0.912	0.412	0.529
2026-04-06	Late-season week	15	1.000	0.889	0.968
2026-04-21	S9 launch	30	0.949	0.936	0.800
2026-04-28	Direct post-S9 week	3	0.857	0.000	0.000

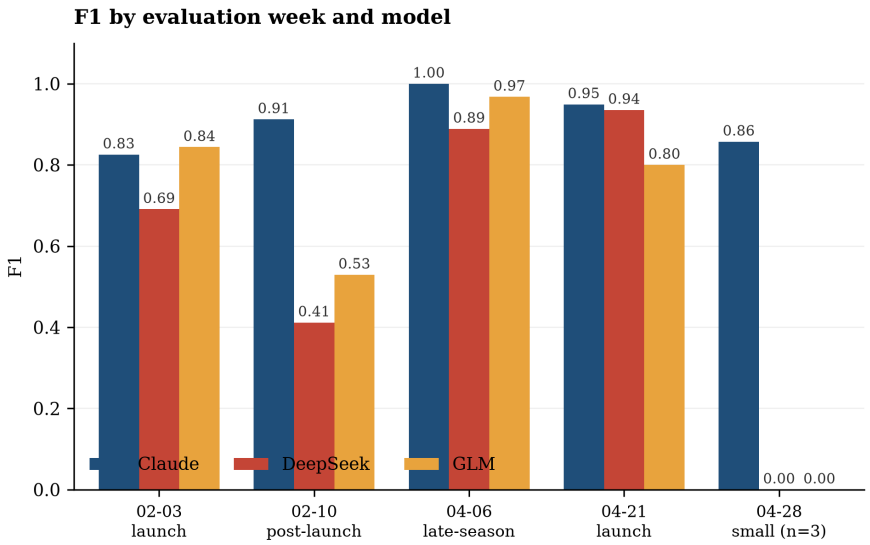

Figure 10: F1 by model across the five Experiment 3 evaluation weeks.

The differences between the models were smaller during the two launch weeks and the late-season week than during the direct post-launch periods. On the S8 launch week, Claude and GLM achieved F1 scores of 0.825 and 0.844, while DeepSeek reached 0.691. During the S9 launch week, Claude and DeepSeek achieved similar scores of 0.949 and 0.936, while GLM reached 0.800. All three models also performed relatively strongly during the late-season week, where their F1 scores ranged from 0.889 to 1.000. These results show that the strongest-performing model was not identical in every period, although Claude remained comparatively strong across all three of these weeks.

The largest separation occurred during the direct post-S8 week. Claude achieved an F1 of 0.912, compared with 0.529 for GLM and 0.412 for DeepSeek. Claude recovered all 26 reference records and produced five false positives. GLM recovered 18 reference records but produced 24 false positives, while DeepSeek recovered 14 and produced 28 false positives. The lower F1 scores for GLM and DeepSeek therefore reflected both missed anomalies and substantial over-reporting.

The week beginning on 2026-04-28 contained only three verified reference anomalies. Claude detected all three and produced one false positive, resulting in an F1 of 0.857. DeepSeek returned two records and GLM returned seven, but neither model recovered any of the three verified anomalies. Both therefore received an F1 of 0. Because the reference set contained only three records, the result is highly sensitive to a small number of correct or incorrect predictions and should not be interpreted in isolation. It is nevertheless retained in the aggregate comparison because all models were evaluated under the same conditions and the period formed part of the predefined benchmark.

6.3.3 Precision, Recall, and False-Positive Control

The aggregate results also show that the models differed in how they balanced precision and recall. Claude achieved pooled precision of 0.891 and recall of 0.925, indicating that it recovered most reference anomalies while producing relatively few additional records. DeepSeek achieved precision of 0.667 and recall of 0.698. GLM achieved similar precision of 0.661 but higher recall of 0.792. Claude therefore maintained the strongest balance between recovering verified anomalies and avoiding unsupported findings. GLM recovered more reference records than DeepSeek, with 84 true positives compared with 74, but also produced more false positives, with 43 compared with 37. The aggregate F1 difference between GLM and DeepSeek reflects this trade-off rather than a uniform advantage across both precision and recall.

The direct post-S8 week provides the clearest example of this behavior. Both DeepSeek and GLM returned 42 anomaly records against a reference set of 26. However, the membership of the returned sets differed.

DeepSeek produced 14 true positives and 28 false positives, whereas GLM produced 18 true positives and 24 false positives. Claude returned 31 records, of which 26 were true positives and five were false positives. The difference between the models was therefore not only how many records they returned, but also how closely those records corresponded to the verified reference set.

6.3.4 Output-Volume Calibration

Figure 11 compares the number of records returned by each model with the number of verified reference records in the corresponding week. Points on the diagonal represent equal predicted and reference-set sizes. This does not measure accuracy by itself, but it shows whether a model tends to over-report or under-report findings.

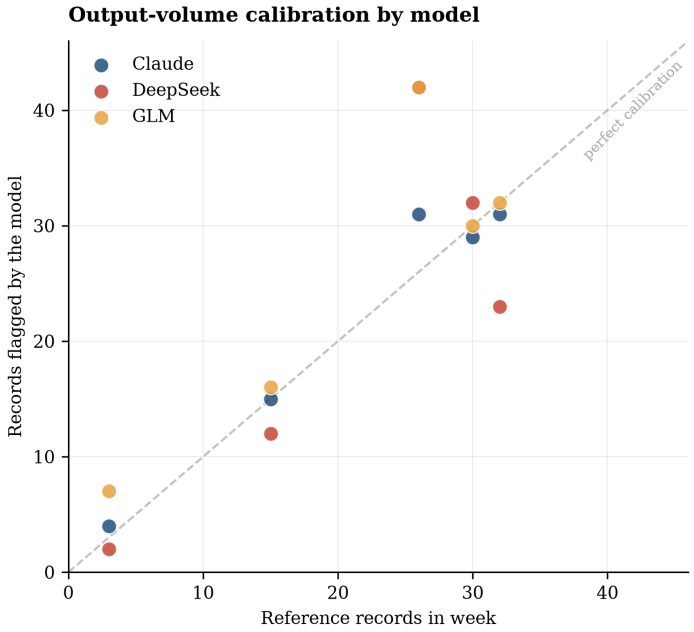

Figure 11: Number of anomaly records returned by each model compared with the number of verified reference records in each week.

Claude's output volume remained relatively close to the reference-set size across all five periods. Its largest difference occurred during the direct post-S8 week, where it returned 31 records against 26 reference anomalies. In the remaining weeks, its output differed from the reference-set size by no more than one record.

DeepSeek and GLM showed larger deviations in particular periods. Both returned 42 records during the direct post-S8 week, which was 16 more than the 26-record reference set. DeepSeek also returned 23 records against 32 during the S8 launch week, while GLM returned seven records against three during the direct post-S9 week. These results are consistent with the lower precision observed for both models, particularly during the direct post-S8 period. However, output volume alone does not show whether the correct anomalies were identified. For example, DeepSeek returned two records during the week containing three verified anomalies, but neither matched the reference set.

Overall, Experiment 3 shows that guided anomaly-detection performance differed across the three evaluated LeoX model deployments. Claude produced the strongest aggregate performance and remained comparatively strong across every evaluated period. DeepSeek and GLM achieved competitive results during some launch and late-season weeks but performed less reliably during the direct post-S8 period and the small direct post-S9 week. However, similar to the issue faced previously, the models did not consistently report the exact parameters, thresholds, baseline windows, preprocessing choices, or intermediate detector results used in their analyses. Follow-up requests for these details also did not always produce complete or consistent information. This limited the extent to how much the observed performance differences could be linked to specific analytical implementation choices rather than to the final anomaly selections alone. This transparency gap is discussed

further in the discussion and limitations sections. These findings apply to the specific guided workflow, model deployments, and evaluation periods used in this thesis. Because each model-week combination was evaluated once, the results should be interpreted as a descriptive comparison rather than as a universal ranking of the underlying model families.

7 Discussion

This thesis investigated whether an LLM-based analytics agent can carry out anomaly detection on live-service game KPIs and whether structured methodological and game-specific context improves its performance. The results provide a positive answer to the first part of this question. Across the evaluated periods, the agents were able to inspect the provided KPI data, execute their own analysis, identify point anomalies and longer-term changes, and return structured outputs that could be evaluated against the verified reference set. The findings therefore show that the task is within the practical capabilities of an LLM-based analytics agent. However, the experiments also show that the quality and reliability depend on the context provided to the model, the underlying model deployment, and the type of period being analyzed.

The main contribution of the results is therefore not simply that an LLM can identify some unusual values. A basic model could potentially flag large spikes or drops by inspecting the data directly. The more important finding is that the guided agent was able to carry out a broader end-to-end analytical workflow that required temporal reasoning, baseline construction, interpretation of launch and post-launch behavior, selection of suitable methods, execution of code, and production of structured anomaly records. This is a more demanding task than asking a language model to describe a time series or identify an obvious extreme value from a short sequence. It requires the model to act as an analytics agent and make several connected methodological decisions before producing the final result. This type of evaluation is consistent with recent agent benchmarks, which assess LLMs through their ability to reason, make decisions, use tools, and complete multi-step tasks rather than through text generation alone [19, 20].

7.1 Ability to Perform End-to-End Anomaly Detection

Experiment 1 shows that Claude Opus 4.8 was capable of performing this workflow even without the anomaly detection Skill, but its unguided behavior was not dependable across the evaluated periods. The unguided condition identified 60 of the 146 verified anomaly records and reached a micro F1 of 0.456. This result is important because it shows that the model was not completely unable to perform the task without additional context. It could identify several anomalies, especially during periods where KPI movements were large and visible. However, its outputs varied substantially depending on the target week. In some periods it under-reported anomalies, in others it over-reported them, and in one post-launch week it returned no findings despite the reference set containing 21 verified records.

The unguided result therefore suggests that general LLM reasoning and access to analytical tools are not sufficient by themselves for reliable live-game anomaly detection. The model can write and execute code, but it still needs to decide what historical period should be used, how weekly behavior should be treated, whether launch-related movement is expected, and which forms of anomaly should be considered. Without task-specific context, these decisions were not made consistently. This supports the distinction made earlier in the thesis between tool access and methodological capability. Giving an agent access to Python and data allows it to perform calculations, but it does not guarantee that it will construct the right analysis for the business and temporal setting. This is consistent with agent-benchmark research showing that tool-using performance can still be limited by reasoning, decision-making, and instruction-following failures [19].

At the same time, the unguided model's ability to recover a meaningful subset of the verified records remains a useful finding. The primary objective of the thesis was to test whether an LLM-based agent could even carry out this type of analytical task at all, rather than to assume that it should immediately achieve perfect agreement with a specialized reference framework. From this perspective, even the unguided results demonstrate that an LLM agent can move beyond text generation and perform a real anomaly-detection workflow on production-level KPI data. The guided results then show how much this capability can be improved when the agent is given more

appropriate context.

7.2 Effect of Methodological and Game-Specific Context

The strongest result in the thesis is the improvement produced by the anomaly detection Skill. The guided condition increased micro F1 from 0.456 to 0.886, while precision increased from 0.513 to 0.895 and recall increased from 0.411 to 0.877. The improvement therefore did not come from simply reporting more anomalies. The guided agent both recovered substantially more verified records and produced fewer unsupported findings. It also performed better in eight of the nine scored weeks, which shows that the aggregate improvement was not caused by one unusually strong period.

This result supports the third research subquestion by showing that structured methodological and domain context can substantially improve an agent's anomaly-detection performance. Importantly, the Skill did not contain the reference code, fixed parameter values, step-by-step implementation instructions, or verified anomaly labels. The agent still had to decide how to apply the information, construct the analysis, execute its own code, and determine which anomalies to return. The difference between the conditions was therefore not that the guided model was given the correct outputs or the steps to get the correct outputs. Instead, it was given a better understanding of the problem, including relevant anomaly types, temporal baselines, seasonality, launch effects, post-launch behavior, and potentially useful detection methods.

The largest improvement appeared during post-launch periods. These periods are especially difficult because KPI values may remain elevated after a season release while also beginning to decline from the launch peak. A comparison against only recent values may make the decline look unusually negative, while a comparison against calm historical periods may make the remaining elevated level look unusually positive. Correct interpretation therefore requires an understanding of the game cycle and the selection of an appropriate temporal baseline. The guided condition reached a mean post-launch F1 of 0.923, compared with 0.146 for the unguided condition. This large difference suggests that the Skill was most valuable when the correct interpretation could not be obtained from the size of the movement alone.

The matched-record analysis from the S9 launch week supports the same conclusion. The guided condition did not only detect more of the verified anomalies, it also produced expected values, directions, magnitudes, and anomaly-type interpretations that were more closely aligned with the reference. This indicates that the Skill improved the analytical interpretation behind the findings rather than only changing the number of records returned. The agent was more likely to construct an expectation that reflected the temporal and game-specific context of the KPI.

These findings are consistent with earlier research showing that LLM performance on time-series anomaly detection depends strongly on how the data and task are represented and on the prompting or analytical strategy used [11, 12]. In the current setting, the additional context appears to help the model use its coding and reasoning capabilities more effectively. The Skill can therefore be understood as an expert context layer that reduces the number of important methodological decisions the model has to make from its general knowledge. It does not remove the need for model reasoning, but it helps that reasoning go towards choices that are more suitable for the data and business context.

7.3 Reliability Across Repeated Runs

Experiment 2 examined whether the stronger guided performance remained reasonably stable when the same task was repeated. This is important because a system may achieve a high score in one run but still be difficult to use if repeated executions produce completely different findings. The results show that guided Claude Opus 4.8 was generally reliable across the three evaluated periods. All 15 runs returned valid outputs, and mean F1 ranged from 0.883 during the S8 launch week to 1.000 during the late-season week.

The late-season period was fully reproducible, with all five runs returning the same 15 verified records and no false positives. The direct post-S8 week also remained highly stable, with a mean F1 of 0.967 and 24 of the 26 verified anomalies detected in every run. Greater variation appeared during the S8 launch week. Mean

F1 remained high at 0.883, but pairwise Jaccard similarity fell to 0.761 and only 20 of the 32 verified records were recovered in every run. This shows that stable overall scores do not always mean that the same individual records were returned in each run.

Across the three periods, 59 of 73 verified anomalies were recovered in all five runs. The remaining instability was concentrated mainly in a smaller group of point anomalies during the S8 launch week, particularly for matchmaking success rate, daily revenue, and paying users. By contrast, all verified DAU and new-registration records were recovered in every run. The results therefore do not suggest that the model randomly changes its entire analysis between runs. Instead, most records remained stable, while a smaller number of more difficult findings were detected inconsistently across executions.

In a practical scenario, the observed stability suggests that a single guided run can provide a useful anomaly analysis. However, the exact set of returned records may still vary in more complex periods. In a production workflow for anomaly detection, repeated runs or a simple consensus rule could therefore be explored when greater confidence is required. Records detected across several independent runs could be treated as more stable findings, while records appearing in only one run could be reviewed more carefully. A related idea has been shown to improve performance in other LLM reasoning settings, where multiple reasoning paths are sampled and the most consistent result is selected [21]. However, this approach was not evaluated as a separate deployment strategy in this thesis, and its effectiveness for the anomaly-detection context would require future testing.

Experiment 2 also provides useful context for the single-run experiments. It shows that Claude's guided performance does not depend entirely on one unusually favorable run. At the same time, the experiment does not remove all uncertainty around the single-run results, because repeated-run behavior was tested on only three weeks (73 anomalies) and only for Claude Opus 4.8. The model comparison should therefore be interpreted descriptively, since run-to-run reliability was not measured for DeepSeek or GLM models.

7.4 Differences Across Model Deployments

Experiment 3 shows that the effectiveness of the guided workflow also depends on the underlying model used to execute it. Claude achieved the strongest aggregate result, with a micro F1 of 0.907 and a macro F1 of 0.909. GLM reached a micro F1 of 0.721, while DeepSeek reached 0.682. Claude also produced the strongest balance between precision and recall as it recovered 98 of the 106 anomalies while producing 12 false positives.

The model differences were not identical in every period. GLM slightly outperformed Claude during the S8 launch week, while DeepSeek performed almost as strongly as Claude during the S9 launch week. All three models also performed well during the selected late-season period, which contained more stable KPI behavior than the direct post-launch periods. This shows that no model dominated every individual week. However, Claude remained the most consistently strong deployment across the complete benchmark and performed substantially better during the direct post-S8 period, where both GLM and DeepSeek missed more verified anomalies and produced many additional records.

The large difference during the direct post-S8 week is consistent with the Experiment 1 finding that post-launch interpretation is one of the most difficult parts of the task. The Skill provided the same theoretical and game-specific context to every model, but the models still differed in how effectively they applied the detection. This suggests that structured guidance alone does not make all models equivalent. The agent must still interpret the instructions, select and implement methods, execute code, and make final anomaly decisions. Differences in reasoning, coding reliability, context use, and run behavior can therefore affect the final result even when the same Skill and input data are provided. This is consistent with AgentBench, which found big performance differences across different models when they were evaluated in the same interactive environments. The study linked these differences partly to variation in long-term reasoning, decision-making, and instruction-following ability [19], which may also help explain why the models in this thesis produced different results despite receiving the same Skill, data, and task instructions.

The benchmark should not be interpreted as a universal ranking of Claude, DeepSeek, and GLM. It compares specific deployments available inside LeoX on a defined task and a limited set of Delta Force evaluation periods. Each model-week combination was also evaluated once, and repeated-run variance was not measured

for DeepSeek or GLM. The more appropriate conclusion is therefore that Claude was the most dependable and accurate model for this workflow, instead of framing it as necessarily the best model for anomaly detection in every setting.

7.5 Implications for Live-Game Analytics

From a practical perspective, the results show that an LLM-based analytics agent can support parts of a live-game KPI monitoring workflow. The guided agent was able to process the data, report the use of several analytical methods, and produce structured anomaly records with high agreement against the verified reference. This suggests that an agent could be useful as a first analytical layer that reviews recent KPIs, highlights unusual movements, and prepares a structured set of findings for analysts or operational teams.

The results do not suggest that the agent should fully replace expert review. The reference-validation process showed that operational context remains important, especially during post-launch periods and for metrics with unusual numerical scales. Instead, the results support a human-in-the-loop workflow where the agent performs the initial analytical detection and domain experts validate the operational meaning of the findings. The agent can perform the repetitive work of constructing baselines, executing detection methods, and screening several KPIs, while analysts remain responsible for confirming the operational meaning of the findings and connecting them to internal events, dashboards, or game decisions.

This is still a meaningful improvement over a fully manual process. Without an automated first layer, analysts may need to inspect several dashboards and individual KPI movements before deciding what deserves attention. A guided agent can reduce this initial workload by producing a structured shortlist of candidate anomalies and the analytical context behind them. The analyst can then focus on validation, explanation, and decision-making rather than starting the investigation from raw KPI values.

The Skill-based setup is also practically useful because it separates reusable domain and methodological knowledge from the underlying model. The same general guidance can be provided to different supported models and updated when the monitoring process changes. However, Experiment 3 shows that the presence of the Skill does not guarantee equal performance across models. Model selection remains important, and the workflow may need model-specific testing before being used in a production setting.

7.6 Answer to the Research Questions

Overall, the findings provide clear answers to the main research question and its sub-questions. The first sub-question was addressed by developing and documenting a suitable combination of residual-based, contextual, multivariate, and changepoint methods for the Delta Force KPI setting. The findings support the suitability of this framework for the current study, but they do not show that it is the best possible approach as that was not the main point. The second sub-question was answered by showing that an LLM-based analytics agent can carry out this task end-to-end, although its unguided performance was inconsistent across different periods. The third sub-question was addressed by establishing that methodological and game-specific context largely improved the agent's accuracy and produced generally stable results across repeated Claude runs. Finally, the fourth sub-question was answered through the model comparison, which showed that performance depends on the underlying deployment, with Claude achieving the strongest overall results in the selected LeoX workflow.

7.7 Interpretation of the Evaluation

The reported precision, recall, and F1 scores should be interpreted as agreement with the verified Delta Force reference set. The reference was built using a reproducible statistical framework and then validated by multiple Tencent analysts and domain experts. This creates a stronger evaluation basis than relying only on whether the LLM output appears plausible. At the same time, anomaly detection contains some degree of contextual judgment, and not every statistically unusual movement has one universally correct interpretation.

The matching procedure used in this thesis is intentionally strict. A record is counted as correct only when the date, metric, and anomaly type match the verified reference. An LLM may therefore identify the correct

KPI movement but use a different anomaly type or assign a different date, in which case it does not receive credit under the main score. Similarly, some unmatched findings may still represent statistically unusual movements even though they were not included in the ground truth set. These cases were not manually reclassified after the experiments, because doing so separately for each output would reduce comparability and introduce bias.

The quantitative results therefore measure how closely the model outputs matched the verified ground truth set under the fixed scoring rules. They may not capture every partially correct or potentially useful finding produced by the models. The agent outputs sometimes contained additional reasoning, intermediate interpretations, or plausible findings that were difficult to compare systematically because the level of detail differed between runs and models. It was also not always possible to recover the exact parameters, baseline windows, intermediate calculations, or implementation choices used by the agents. A more extensive qualitative analysis could help provide more insight into these decisions, but it would require more standardized logging and substantially more manual review.

This does not weaken the main conclusion of the thesis. The central question was whether an LLM-based analytics agent could carry out the end-to-end anomaly-detection task at a meaningful level and whether structured methodological context improved that ability. The evaluation showed that the guided Claude run reproduced most of the verified findings and remained stable across repeated runs. Claude also achieved the strongest overall performance in the model benchmark. These findings provide evidence that guided LLM agents can perform useful anomaly detection on live-service game KPIs, while also showing that they are most dependable when supported by clear context, an appropriate model, and continued human validation.

The following section discusses the limitations that define how far these findings can be generalized and the future work needed to test the workflow across wider datasets, models, metrics, and deployment conditions.

8 Limitations and Future Work

The findings of this thesis provide evidence that guided LLM-based analytics agents can perform useful anomaly detection on live-service game KPIs. However, the conclusions should be interpreted within the scope of the data, reference framework, models, and experimental conditions used in the research. The following limitations define how far the findings can currently be generalized and identify the most important directions for future work.

8.1 Scope and Generalization

This research is limited to one live-service game and a selected set of KPIs. Although many live-service games experience similar patterns around updates, launches, and post-launch decay, the findings may not generalize directly to games with different genres, platforms, player behavior, monetization models, or operational structures. The evaluated metrics also represent only part of the wider game-health monitoring problem, as other relevant measures could include retention, session length, churn, conversion rate, latency, crash rate, queue time, and regional or platform-specific performance. Future work should therefore test the framework and guided-agent workflow on additional games, longer time periods, and a wider range of business and technical metrics to determine whether the conclusions remain stable outside the Delta Force setting.

8.2 Reference Set and Evaluation Design

Because the Delta Force dataset did not contain labeled anomalies, the thesis constructed a statistical reference framework and then had its candidate anomalies validated by Tencent analysts and domain experts. This created a stronger evaluation basis than relying only on an automated detector or on whether the LLM outputs appeared plausible. Approximately 90% of the original candidate records were retained, and the reviewers did not identify relevant anomalies that had been missed by the framework. Nevertheless, the final reference set still includes an element of expert judgment. Each evaluation period was reviewed by one expert rather than being independently labeled by several reviewers. Although the reviewers reported that the framework had not missed relevant anomalies within the periods and internal context they examined, this cannot be interpreted as a measured recall estimate because no separate set of independent labels was produced. Agreement between reviewers also could

not be calculated across the complete reference set. Future work could strengthen the validation process by having multiple experts independently label the same periods before comparing their judgments and resolving disagreements through formal adjudication.

The main evaluation also uses a strict matching rule based on date, metric, and anomaly type. This keeps the comparison consistent and prevents the results from being changed after reviewing the outputs. However, it does not capture every form of partial agreement. A model may identify the correct KPI movement but assign a different anomaly type or date, in which case it does not receive credit under the main score. Some unmatched outputs may also represent plausible or borderline statistical movements that were not included in the verified reference set. The reported precision, recall, and F1 values should therefore be interpreted as exact agreement with the verified reference rather than as a complete measure of every useful observation produced by the models. Future work could add a secondary qualitative or graded evaluation that gives partial credit for closely related findings or reviews unmatched records separately. This should remain secondary to the fixed reference-based score so that the main comparison remains reproducible.

8.3 Experimental Coverage and Model Reliability

Experiment 1 evaluated nine anomaly weeks and one no-anomaly control week. Experiment 2 repeated the guided Claude setup five times across three selected periods, while Experiment 3 compared three model deployments across five anomaly weeks. These experiments provide a meaningful number of record-level comparisons, but they do not cover every possible game period or source of model variation. The repeated-run experiment was also limited to Claude Opus 4.8. It showed relatively stable overall performance, although record-level agreement was lower during the S8 launch week. DeepSeek and GLM were evaluated once per model-week combination, so their run-to-run variation remains unknown. The Experiment 3 results should therefore be interpreted as a descriptive comparison of the selected LeoX models for this context alone, rather than as a final ranking of the underlying model families.

Future work should repeat every model condition across multiple independent runs and include more launch, post-launch, mid-season, late-season, and no-anomaly periods. This would make it possible to estimate both average performance and variance for each model. It would also show whether the observed difficulty of post-launch and launch periods remains consistent across a wider benchmark. Some metrics also created more difficult edge cases than others. Matchmaking failure rate is close to zero, so small absolute changes can appear very large in relative terms. The framework uses denominator floors and metric-specific filtering to reduce this effect, but the metric still had many inconsistent detections and false positives in the repeated-run results. Future work could use more metric-specific calibration, such as absolute percentage-point changes, confidence intervals based on underlying traffic volume, or separate rules for count metrics and bounded rate metrics.

8.4 Transparency and Deployment

A further limitation is that the LLM outputs did not always provide complete information about how the analysis had been implemented. The models often reported using relevant detection methods, but they did not consistently document the exact parameters, baseline windows, pre-processing choices, detector outputs, or generated code used in each run. Follow-up prompts were used to request these details, but the resulting explanations were not always complete or consistent. This means that the evaluation can measure the quality of the final anomaly records, but it cannot fully explain why a model detected or missed every individual anomaly. Two runs may report using the same method while applying it with different parameters or historical windows. Similarly, differences between models may arise from method selection, code generation, data handling, or final interpretation, but the available outputs do not always allow these factors to be separated.

Future work should require each run to save a standardized log file that contains all of the generated code, baseline periods, parameters, intermediate detector results, expected values, and final consolidation logic. This would make it possible to audit the full process rather than only the final output. It would also allow for a more detailed comparison of where performance differences between models originate. The repeated-run results also suggest that findings detected across several independent runs could be assigned greater confidence than those appearing only once. This could help reduce unstable outputs during difficult periods, although the effectiveness

of this approach would need to be evaluated separately.

Overall, the main limitations concern generalization, experimental scale, anomaly edge cases, and limited transparency into the agents' analytical execution. Within the evaluated Delta Force setting, however, the results still show that guided LLM-based analytics agents can reproduce most verified anomalies and carry out a meaningful end-to-end anomaly-detection workflow. The next stage is therefore to determine how reliably this approach can be scaled and possibly even integrated into a live monitoring environment.

9 Personal Contribution and Practical Implementation

Beyond discussing the experiments and research findings, I also wanted to show how this work has been used in practice. I used the findings from the thesis to build a practical anomaly-detection layer on top of LeoX within Tencent's internal environment. The system can query KPI data from different games and apply the methods and analytical principles developed during the research. This means that the work did not remain limited to the Delta Force experiments, but was turned into a reusable workflow that can support other game analytics tasks.

The layer is also already running in production. Scheduled cron jobs run the workflow every week and generate HTML anomaly reports for the team to review and validate the findings before they are included in the wider reporting process and used in executive dashboards. This creates a clearer process from raw KPI data to anomaly detection, validation, reporting, and business communication.

Before this workflow was introduced, anomaly analysis relied more heavily on manually checking dashboards and individual KPI movements. The new layer makes this process more efficient by automatically preparing a focused set of unusual movements for review. Analysts can therefore spend less time searching across all available metrics and more time investigating the most relevant findings, linking them to game events, and deciding what should be communicated.

The reports are also useful for executives and wider business teams. Instead of presenting the findings only as raw structured outputs, the HTML reports highlight which movements were unusual, how they compared with expected behavior, and which areas may require further investigation. The validated findings can then support executive dashboards, presentations, weekly reporting, and broader game-performance analysis.

My contribution was therefore not only to evaluate LLM-based anomaly detection, but also to turn the research into a working analytical layer within Tencent. The thesis provided the methodological foundation, while the implemented workflow made the approach repeatable and usable for real monitoring and decision-making across games.

10 Conclusion

This thesis showed that LLM-based analytics agents can carry out meaningful anomaly detection on live-service game KPIs, but that their performance depends strongly on how the task is structured. Without specific guidance, the agent was able to identify some relevant anomalies, but its results were inconsistent across different periods. Providing methodological and game-specific context largely improved accuracy, reduced unsupported findings, and produced generally stable results across repeated Claude runs. The model comparison also showed that the underlying deployment remained important, with Claude achieving the strongest overall performance in the selected LeoX workflow.

The results are especially encouraging because it was initially unclear whether an LLM-based analytics agent could even carry out this type of anomaly-detection task at a meaningful level. The agents were not given the verified anomaly records or the exact reference implementation, but still had to analyze the data, construct suitable baselines, select methods, execute code, and interpret the final findings. Although the evaluation was limited to one game, a selected set of KPIs, and a relatively small number of repeated runs, the findings provide

evidence that guided LLM agents can support a real anomaly-monitoring process.

The practical implementation developed during the thesis further shows that this work can be applied beyond the experimental setting. The resulting workflow is already being used to generate structured anomaly reports for analyst review and wider reporting. This provides a foundation for more consistent KPI monitoring methods while keeping expert validation as an important part of the process.

Overall, the thesis provides an encouraging starting point for further research into LLM-based anomaly detection in live-game analytics. Future work should test the approach across more games, metrics, models, and continuously running monitoring environments, while improving transparency into how the agents perform their analysis. It should also strengthen the construction and validation of the ground truth through broader expert review, independent labelling, and closer examination of borderline or unmatched anomalies. With further testing and refinement, guided LLM agents could become a valuable additional layer in production analytics and decision-support workflows.

References

[1] McKinsey & Company, "The State of AI in Early 2024: Gen AI Adoption Spikes and Starts to Generate Value." https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-in-early-2024-gen-ai-adoption-spikes-and-starts-to-generate-value, 2024. Accessed: 2026-07-15.

[2] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[3] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–33, 2021.

[4] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "Stl: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.

[5] J. Hochenbaum, O. S. Vallis, and A. Kejariwal, "Automatic anomaly detection in the cloud via statistical learning," 2017.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422, IEEE, 2008.

[7] R. Killick, P. Fearnhead, and I. A. Eckley, "Optimal detection of changepoints with a linear computational cost," *Journal of the American Statistical Association*, vol. 107, no. 500, pp. 1590–1598, 2012.

[8] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[9] K. H. Kim and H. K. Kim, "Oldie is goodie: Effective user retention by in-game promotion event analysis," 2019.

[10] A. R. Kang, S. H. Jeong, A. Mohaisen, and H. K. Kim, "Multimodal game bot detection using user behavioral characteristics," 2016.

[11] S. Alnegheimish, L. Nguyen, L. Berti-Equille, and K. Veeramachaneni, "Large language models can be zero-shot anomaly detectors for time series?," 2024.

[12] M. Dong, H. Huang, and L. Cao, "Can llms serve as time series anomaly detectors?," 2024.

[13] Z. Zhou and R. Yu, "Can llms understand time series anomalies?," 2024.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[15] Steam Charts, "Delta Force - Steam Charts." https://steamcharts.com/app/2507950, 2026. Accessed: 2026-07-09.

[16] Anthropic, "Models Overview." https://platform.claude.com/docs/en/about-claude/models/overview, 2026. Accessed: 2026-07-14.

[17] DeepSeek-AI, "DeepSeek-V4-Pro Model Card." https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro, 2026. Accessed: 2026-07-14.

[18] GLM-5 Team, "GLM-5: From Vibe Coding to Agentic Engineering." arXiv preprint arXiv:2602.15763, 2026. Available at https://arxiv.org/abs/2602.15763.

[19] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, *et al.*, "Agentbench: Evaluating llms as agents," *arXiv preprint arXiv:2308.03688*, 2023.

[20] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, Y. Li, Z. Liao, C. Wei, Z. Lu, M. Xue, *et al.*, "Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery," *arXiv preprint arXiv:2410.05080*, 2024.

[21] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.

A Appendix

A.1 Framework Parameters

Tables 9 and 10 present the complete parameter configuration used by the reference framework. The parameter values were held constant across all evaluated target periods. Random seeds were also fixed so that repeated framework executions remained reproducible.

Table 9: Parameters used for expected-value construction and point-anomaly detection.

Component	**Parameter**	**Value or setting**
STL decomposition	Seasonal period	7 days, representing the weekly cycle in the daily KPI data.
	Robust fitting	Enabled.
	Minimum history	56 days, corresponding to approximately eight weekly cycles.
	Lag-7 autocorrelation gate	STL was applied when $\rho_7 > 0.30$.
Expected-value construction	Trend projection	Linear slope estimated from the final 14 fitted trend values.
	Seasonal projection	Most recent fitted weekly seasonal cycle.
	Fallback baseline	7-day rolling median when the STL conditions were not met.
Z-score detector	Medium threshold	$\lvert z\rvert \geq 2.5$.
	Critical threshold	$\lvert z\rvert \geq 3.0$.
Univariate Isolation Forest	Contamination	0.02.
	Random seed	42.
Multivariate Isolation Forest	Input	Residual matrix containing all monitored metrics.
	Attribution	Metrics with the largest standardized residual contributions.
Magnitude calibration	Medium threshold	90th percentile of absolute deviations observed during calm periods.
	Critical threshold	95th percentile of absolute deviations observed during calm periods.
	Launch relaxation factor	Calm-period thresholds multiplied by 1.5 during launch periods.

Table 10: Parameters used for changepoint detection, season comparison, and reproducibility.

Component	**Parameter**	**Value or setting**
PELT regime-shift detection	Cost model	`l2`, representing changes in the mean level.
	Penalty	3.
	Minimum segment size	7 days.
CUSUM drift detection	Slack parameter, k	0.5.
	Decision threshold, h	4.0.
Changepoint layer	Minimum history	90 days.
	Onset attribution	A changepoint was assigned to a target period when its estimated onset occurred within that period.
Season comparison	Reference period	Same launch-relative day in the immediately preceding season.
Candidate consolidation	Minimum detector support	At least two detection signals were required for the same date and metric before a point-anomaly candidate was recorded.
Reproducibility	Random seeds	NumPy generator seed 0 and Isolation Forest seed 42.

A.2 Summary of Skill Content

Table 11: Summary of the contextual information provided through the experimental anomaly-detection Skill.

Context category	**Description**
Anomaly concepts	Point anomalies, contextual anomalies, collective anomalies, changepoints, and drift.
Temporal reasoning	The importance of historical baselines, weekly seasonality, trend, and keeping the target period separate from the historical data used to construct expectations.
Game context	The possible effects of season launches, game updates, events, and other operational changes on KPI behavior.
Potential methods	General descriptions of residual analysis, statistical detection, Isolation Forest, season comparison, multivariate checks, and changepoint methods.
KPI information	Definitions and interpretation of the monitored activity, acquisition, monetization, and matchmaking metrics.
Reporting requirements	Instructions to return each detected anomaly in a consistent, machine-readable JSON structure.
Explicitly excluded	Code, exact parameter values, fixed baseline calculations, step-by-step implementation instructions, verified anomaly labels, expected values, and correct outputs for the evaluation periods.

A.3 Experimental Prompt Templates

The following templates are the guided and unguided prompts used in the experiments.

A.3.1 Guided Condition Prompt

Run an anomaly-detection analysis for Delta Force Mobile for the period `[TARGET_START]` to `[TARGET_END]`, inclusive.

Use the provided anomaly-detection Skill and read all of its associated files before beginning the analysis.

Use only the attached CSV dataset. Do not query the database or retrieve additional data. The file contains the full available daily KPI series, including observations before and after the target period.

Analyze the following metrics:

- `dau`;
- `daily_revenue`;
- `paying_users`;
- `new_registrations`;
- `matchmaking_success_rate`; and
- `matchmaking_failure_rate`.

Use the metric definitions, season calendar, directionality rules, deviation floors, temporal context, and anomaly-detection guidance contained in the Skill. Independently determine how this guidance should be applied to the supplied data.

Return the detected anomalies as valid JSON using the required reporting structure like this: {"anomalies": [{...}, {...}, ...]}. Produce one record for each unique combination of `date`, `metric`, and `anomaly_type`.

Return only detected anomaly records. Do not include normal observations, summaries, or explanatory text outside the JSON structure. If no anomalies are detected, return an empty anomaly list.

A.3.2 Unguided Condition Prompt

Run an anomaly-detection analysis for data attached for the period `[TARGET_START]` to `[TARGET_END]`, inclusive.

Use only the attached CSV dataset. Do not query the database or retrieve additional data. The file contains the full available daily KPI series, including observations before and after the target period.

Analyze the following metrics:

- `dau`;
- `daily_revenue`;
- `paying_users`;
- `new_registrations`;
- `matchmaking_success_rate`; and
- `matchmaking_failure_rate`.

Independently determine an appropriate analytical approach for identifying anomalies in the supplied live-game KPI data. Generate and execute code where necessary.

Return the detected anomalies as valid JSON using the required reporting structure like this: {"anomalies": [{...}, {...}, ...]}. Produce one record for each unique combination of `date`, `metric`, and `anomaly_type`.

Return only detected anomaly records. Do not include normal observations, summaries, or explanatory text outside the JSON structure. If no anomalies are detected, return an empty anomaly list.

A.4 Required Anomaly Reporting Format

Both conditions returned detected anomalies using a standardized JSON structure. Each record identified the date, metric, anomaly type, observed and expected values, deviation, direction, magnitude, and a brief detection note. An example is shown below.

```
{
  "anomalies": [
    {
      "date": "YYYY-MM-DD",
      "metric": "daily_revenue",
      "anomaly_type": "point",
      "observed": 1056814.9,
      "expected": 576729.8,
      "deviation_pct": 83.2,
      "direction": "above",
      "magnitude": "extreme",
      "detection_note":
        "Large positive deviation relative to the
        relevant historical baseline."
    }
  ]
}
```